

بسم الله الرحمن الرحيم



دانشگاه صنعتی اصفهان

دانشکده مهندسی برق و کامپیوتر

بررسی امکان بهبود بازشناسی گفتار فارسی با ترکیب خروجی مدل‌های موجود
**Investigating The Possibility of Improving Persian
Automatic Speech Recognition by Combining Outputs of
Existing Models**

گزارش پروژه کارشناسی

زهرا توکلی

گرایش: سیستم‌های هوشمند

استاد راهنما

دکتر زینب مالکی

تابستان ۱۴۰۵

تشکر و قدردانی

پروردگار منّان را سپاسگزارم

فهرست مطالب

صفحه	عنوان
پنج	فهرست مطالب
نه	فهرست تصاویر
۱	چکیده
۲	فصل اول: مقدمه
۲	۱-۱ زمینه و اهمیت
۳	۲-۱ بیان مسئله
۳	۳-۱ ایده اصلی و پرسش پژوهش
۴	۴-۱ راه حل پیشنهادی
۴	۱-۴-۱ هم ترازی کاراکتری
۴	۲-۴-۱ سطح پردازش: کاراکتر در برابر زیرواژه
۵	۳-۴-۱ رمزگذار و ادغام گیت دار و رمزگشا
۵	۵-۱ ساختار گزارش
۶	فصل دوم: کارهای مرتبط
۶	۱-۲ مقدمه
۶	۲-۲ تصحیح پس از تشخیص گفتار: رویکردهای عمومی
۷	۳-۲ تصحیح پس از ASR برای زبان فارسی
۸	۴-۲ ترکیب چندنمایی و سیستم های ترکیبی
۸	۵-۲ تصحیح خطای OCR و ارتباط آن با تصحیح ASR
۹	۶-۲ مدل های زبانی بزرگ در تصحیح پس از ASR
۱۰	۷-۲ جمع بندی
۱۱	فصل سوم: روش شناسی
۱۱	۱-۳ فرمول بندی مسئله
۱۲	۲-۳ معماری مدل های ASR پایه
۱۲	۱-۲-۳ مدل Whisper
۱۲	۲-۲-۳ مدل MMS

۳-۳	پیش‌پردازش و هم‌ترازی ورودی	۱۲
۳-۳-۱	راهبرد نخست — هم‌ترازی با کاراکتر تهی	۱۳
۳-۳-۲	راهبرد دوم — هم‌ترازی با استفاده از کاراکتر دنباله طولانی‌تر	۱۳
۴-۳	معماری مدل پیشنهادی	۱۴
۴-۳-۱	رمزگذار مشترک	۱۴
۴-۳-۲	مکانیزم ادغام گیت‌دار	۱۵
۴-۳-۳	رمزگشا با مکانیزم توجه	۱۸
۱۹	فصل چهارم: داده‌ها، نرمال‌سازی و پیاده‌سازی	
۴-۱	مجموعه داده	۱۹
۴-۱-۱	معرفی مجموعه داده اولیه	۱۹
۴-۱-۲	ساختار ستون‌های مجموعه داده	۲۰
۴-۱-۳	تفکیک داده‌ها	۲۰
۴-۱-۴	تولید خروجی‌های ASR	۲۱
۴-۲	نرمال‌سازی متن	۲۱
۴-۲-۱	ضرورت نرمال‌سازی	۲۱
۴-۲-۲	کتابخانه Shekar	۲۲
۴-۲-۳	مراحل نرمال‌سازی با مثال	۲۲
۴-۲-۴	اصلاح خط الرسم اشتباه در مجموعه داده Common Voice	۲۳
۴-۳	پالایش داده‌های نامعتبر	۲۶
۴-۳-۱	ضرورت پالایش	۲۶
۴-۳-۲	نتایج پالایش	۲۶
۴-۳-۳	مجموعه داده نهایی	۲۸
۴-۴	پیاده‌سازی و آموزش	۲۸
۴-۴-۱	پیکربندی مدل	۲۸
۴-۴-۲	لایه تعبیه کاراکتری یا زیرواژه ای	۲۸
۴-۴-۳	دیکشنری کاراکتری	۲۸
۴-۴-۴	تفاوت مدل کاراکتری و زیرواژه‌ای	۲۸
۴-۴-۵	روش‌های کاهش نرخ Teacher Forcing	۲۹
۴-۴-۶	Dropout حسی	۳۰
۴-۴-۷	Dropout در سایر بخش‌ها	۳۰
۴-۴-۸	مقادیر هایپرپارامترها	۳۱
۴-۴-۹	سخت‌افزار و زمان آموزش	۳۱
۴-۴-۱۰	ادخیره‌سازی و بازیابی	۳۱
۴-۵	تحلیل خطا و ماهیت نویز	۳۲

۳۲	۱-۵-۴ ابزارهای تحلیل خطا
۳۲	۲-۵-۴ نتایج تحلیل خطا
۳۲	۳-۵-۴ تحلیل هم‌آوایی در زبان فارسی
۳۴	فصل پنجم: نتایج تجربی
۳۴	۱-۵ مقدمه فصل
۳۴	۲-۵ نتایج کمی مدل‌ها
۳۴	۱-۲-۵ خطوط پایه
۳۵	۲-۲-۵ مدل زیرواژه‌ای
۳۶	۳-۲-۵ مدل کاراکتری با هم‌ترازی کاراکتر تهی
۳۸	۴-۲-۵ مدل کاراکتری با هم‌ترازی رشته طولانی‌تر
۳۹	۵-۲-۵ مدل کاراکتری با فرمول گیت‌های مقاله و Gate Variance
۴۰	۶-۲-۵ جدول جامع مقایسه
۴۰	۳-۵ تحلیل فروپاشی گیت
۴۰	۱-۳-۵ تعریف و مکانیزم فروپاشی گیت
۴۲	۲-۳-۵ شواهد تجربی فروپاشی در سه مدل
۴۳	۳-۳-۵ Gate Variance Regularization: فرمول‌بندی کامل
۴۴	۴-۳-۵ توضیح Label Smoothing
۴۵	۵-۳-۵ سایر اصلاحات برای تقویت پویایی گیت
۴۶	۴-۵ ماهیت موقعیتی گیت
۴۶	۱-۴-۵ طرح آزمون و فرمول‌بندی
۴۷	۲-۴-۵ نتایج همبستگی خام
۴۷	۳-۴-۵ تحلیل همبستگی جزئی — کشف همبستگی کاذب
۴۷	۴-۴-۵ تفسیر نهایی و نتیجه‌گیری آزمون
۴۸	۵-۵ تحلیل نظری: چرا معماری برای فارسی نامناسب است؟
۴۸	۱-۵-۵ توانایی نظری معماری
۴۸	۲-۵-۵ سه مانع عملی
۴۹	۳-۵-۵ تفاوت بنیادی با خط دواناگری
۴۹	۶-۵ مقایسه با مقاله مرجع
۵۰	۷-۵ جمع‌بندی
۵۲	فصل ششم: جمع‌بندی و پیشنهاد کارهای آینده
۵۲	۱-۶ مقدمه فصل
۵۲	۲-۶ دستاوردهای فنی
۵۳	۳-۶ محدودیت‌های پژوهش
۵۴	۴-۶ پیشنهاد کارهای آینده

۵۴	۱-۴-۶ استفاده از مدل‌های زبانی بزرگ
۵۴	۲-۴-۶ ویژگی‌های واجی
۵۵	۳-۴-۶ معماری ترنسفورمر
۵۵	۵-۶ پیوست ریاضی: تحلیل کران اطلاعاتی
۵۵	۱-۵-۶ ظرفیت اطلاعاتی BiLSTM
۵۵	۲-۵-۶ اطلاعات لازم برای نگارش صحیح واژگان فارسی
۵۶	۳-۵-۶ مقایسه و نتیجه‌گیری
۵۷	مراجع

فهرست تصاویر

- ۱-۵ نمودار گیت‌های مدل زیرواژه‌ای. نمودار سمت چپ توزیع مقادیر گیت را نشان می‌دهد و نمودار سمت راست تغییرات میانگین گیت با موقعیت کاراکتر در جمله را نمایش می‌دهد. ۳۶
- ۲-۵ توزیع خطاهای مدل زیرواژه‌ای در چهار پنل: توزیع CER، توزیع خطاهای سطح کلمه، عملیات ویرایش در سطح کاراکتر، و رابطه CER با طول جمله. ۳۶
- ۳-۵ نتایج CER و WER مدل زیرواژه‌ای روی مجموعه تست در مقایسه با خطوط پایه. ۳۷
- ۴-۵ نمودار گیت‌های مدل کاراکتری با کاراکتر تهی. در این مدل، توزیع گیت‌ها پهن‌تر از مدل زیرواژه‌ای است که نشان‌دهنده پویایی بیشتر مکانیزم گیت است. ۳۸
- ۵-۵ توزیع خطاهای مدل کاراکتری با کاراکتر تهی. مقایسه این نمودار با شکل ۲-۵ نشان می‌دهد که مدل کاراکتری خطاهای کمتری در همه دسته‌ها تولید کرده است. ۳۸
- ۶-۵ نتایج CER و WER مدل کاراکتری با کاراکتر تهی روی مجموعه تست. ۳۹
- ۷-۵ نتایج CER و WER مدل کاراکتری با فرمول مقاله و Gate Variance. ۴۰
- ۸-۵ نتایج CER و WER مدل کاراکتری با فرمول ترکیب محدب و Gate Variance. ۴۰

چکیده

مدل‌های پیشرفته تشخیص گفتار خودکار (ASR) مانند Whisper و MMS، با وجود عملکرد موفق در زبان‌های پرمنیع مانند انگلیسی، در مواجهه با زبان‌های کم‌منیع و به‌ویژه در شرایطی چون گفتار محاوره‌ای، تنوع لهجه‌ها و ضبط غیرایده‌آل، دچار افت دقت می‌شوند. زبان فارسی نیز به‌عنوان یک زبان با منابع محدود برچسب‌گذاری شده است و از این چالش مستثنا نیست؛ به‌گونه‌ای که خروجی مدل‌های ASR چندزبانه روی آن حاوی خطاهای نیمه سیستماتیک و تکرارشونده است. بررسی‌ها نشان می‌دهد خطاهای مدل‌های مختلف ASR در بسیاری از موارد مکمل یکدیگرند؛ به این معنا که آنچه یک مدل نادرست بازشناسی می‌کند، ممکن است در خروجی مدل دیگر به‌درستی ثبت شده باشد. این پروژه با تکیه بر همین مشاهده، یک چارچوب تصحیح پس از تشخیص گفتار (Post-ASR Correction) را برای زبان فارسی بررسی می‌کند که در آن خروجی دو مدل Whisper و MMS به‌عنوان دو «نما» از یک ورودی صوتی واحد در نظر گرفته شده و با ترکیب چندنمایی، انتظار می‌رود دقت بازشناسی نهایی ارتقا یابد.

معماری پیشنهادی از سه مؤلفه اصلی تشکیل شده است: نخست، هم‌ترازی کاراکتری دو خروجی ASR با الگوریتم فاصله‌ی لُونشتاین که امکان مقایسه‌ی حرف‌به‌حرف دو نما را در طول یکسان فراهم می‌کند. دوم، یک رمزگذار مشترک مبتنی بر BiLSTM که بردارهای تعبیه‌شده‌ی دو نما را دریافت کرده و به‌همراه یک بلوک ادغام گیت‌دار (Gated Fusion) با گیت‌های متقاطع (Cross-view Gates)، در هر گام زمانی به‌صورت پویا وزن هر نما را تعیین می‌کند؛ به این ترتیب مدل می‌تواند در هر موقعیت کاراکتری، به نمای مطمئن‌تر اتکا کند. سوم، یک رمزگشای LSTM یک‌جهته مجهز به مکانیزم توجه (Attention) از نوع Bahdanau که متن نهایی متناظر با فایل صوتی را تولید می‌کند. به‌منظور افزایش مقاومت مدل در برابر خطاهای ورودی و کاهش خطر بیش‌برازش، از تکنیک‌های افزایش داده مانند حذف تصادفی کلمات در داده‌آموزشی نیز بهره گرفته شده است. ارزیابی سامانه پیشنهادی روی مجموعه داده استاندارد گفتار فارسی Common Voice و با معیارهای نرخ خطای کاراکتر (CER) و نرخ خطای کلمه (WER) انجام شده است.

واژه‌های کلیدی: ۱- Post-ASR Correction، ۲- MMS، ۳- Whisper، ۴- CER، ۵- WER، ۶- BiLSTM، ۷- Encoder، ۸-

Gated Fusion، ۹- Decoder، ۱۰- Attention

فصل اول

مقدمه

۱-۱ زمینه و اهمیت

تشخیص گفتار خودکار (Automatic Speech Recognition, ASR) در سال‌های اخیر با ظهور مدل‌های مبتنی بر یادگیری عمیق و به‌ویژه معماری‌های ترنسفورمر، پیشرفت چشمگیری داشته است. مدل‌های چندزبانه‌ای مانند Whisper و MMS با آموزش روی حجم عظیمی از داده‌های صوتی چندزبانه، توانسته‌اند در زبان‌های پرمنبع (High-Resource) مانند انگلیسی، چینی یا اسپانیایی به دقت‌های بسیار بالایی دست یابند. در مقابل، زبان کم‌منبع (Low-Resource) به زبانی گفته می‌شود که در مقایسه با این زبان‌ها، منابع دیجیتال، پیکره‌های متنی و صوتی برچسب‌گذاری‌شده، و ابزارهای پردازش زبان طبیعی برای آن به‌مراتب محدودتر است. این کمبود منابع، آموزش مدل‌های یادگیری عمیق را که به حجم عظیمی از داده برچسب‌خورده نیاز دارند با چالش جدی مواجه می‌کند و همین امر باعث می‌شود مدل‌های ASR چندزبانه، حتی وقتی روی زبان‌های پرمنبع به دقت‌های بالا می‌رسند، در برخورد با زبان‌های کم‌منبع دچار افت محسوس عملکرد شوند.

زبان فارسی با بیش از صد میلیون گویشور، علی‌رغم جایگاه فرهنگی و جغرافیایی گسترده‌اش، در حوزه فناوری‌های گفتاری همچنان در دسته زبان‌های کم‌منبع قرار می‌گیرد. کمبود داده‌های صوتی برچسب‌گذاری‌شده، تنوع گسترده لهجه‌ها و گویش‌ها، تفاوت‌های قابل‌توجه میان گفتار رسمی و محاوره‌ای، و پیچیدگی‌های خط

فارسی مانند نویسه‌های چندشکلی (مثلاً «ک/ك»، «ی/ي»)، اعراب‌گذاری، همزه، و فاصله‌گذاری، همگی از عواملی هستند که بازشناسی دقیق گفتار فارسی را به چالشی جدی تبدیل می‌کنند.

۱-۲ بیان مسئله

مدل‌های ASR پیشرفته کنونی مانند Whisper و MMS، در نبود داده آموزشی اختصاصی برای فارسی، خروجی‌هایی تولید می‌کنند که حاوی خطاهای نیمه سیستماتیک و تکرار شونده است. این خطاها معمولاً از جنس جانشینی (Substitution)، درج (Insertion)، حذف (Deletion) هستند.

نکته کلیدی آن است که این خطاها ماهیتی کاملاً تصادفی ندارند و در بسیاری از موارد، الگوهای خطای دو مدل مختلف مکمل یکدیگرند؛ به این معنا که آنچه یک مدل نادرست بازشناسی می‌کند، ممکن است در خروجی مدل دیگر به درستی ثبت شده باشد. ریشه این مکمل بودن را باید در تفاوت معماری و داده آموزشی این دو مدل جست‌وجو کرد: Whisper بر پایه معماری رمزگذار-رمزگشای ترنسفورمر و با نظارت ضعیف روی میلیون‌ها ساعت داده چندزبانه آموزش دیده است، در حالی که MMS بر پایه معماری wav2vec ۰.۲ و با هدف پوشش بیش از هزار زبان طراحی شده است. این تفاوت‌ها در معماری و داده آموزشی باعث می‌شود این دو مدل خطاهای متفاوتی مرتکب شوند و در نتیجه اطلاعات آن‌ها در کنار یکدیگر بتواند تصویر کامل‌تری از گفتار ورودی ارائه دهد. همین مشاهده، انگیزه اصلی استفاده از رویکرد «ترکیب چندنمایی» (Multi-View Fusion) در این پروژه را فراهم می‌کند.

۱-۳ ایده اصلی و پرسش پژوهش

پرسش اصلی این پروژه آن است که:

آیا می‌توان با ترکیب خروجی چند مدل ASR ناهمگون، خطاهای بازشناسی گفتار فارسی را به‌طور مؤثر کاهش داد؟

فرض بنیادین این است که خروجی هر مدل ASR را می‌توان به‌عنوان یک «نما» (View) از سیگنال گفتار در نظر گرفت؛ هر نما حاوی اطلاعاتی است که ممکن است در نمای دیگر مفقود یا مخدوش باشد. اگر مدلی بتواند به‌صورت پویا یاد بگیرد که در هر موقعیت به کدام نما اتکا کند، خروجی نهایی از هر یک از مدل‌های منفرد دقیق‌تر خواهد بود.

۴-۱ راه حل پیشنهادی

در این پروژه، یک چارچوب تصحیح پس از تشخیص گفتار (Post-ASR Correction) برای زبان فارسی پیاده سازی می شود که خروجی دو مدل Whisper و MMS را به عنوان دو نمای موازی دریافت کرده و متن تصحیح شده را تولید می کند. معماری پیشنهادی از سه بخش اصلی تشکیل شده است: نخست، هم ترازی کاراکتری دو خروجی ASR؛ دوم، یک رمزگذار مشترک مبتنی بر BiLSTM به همراه بلوک ادغام گیت دار؛ و سوم، یک رمزگشای مجهز به مکانیزم توجه که متن نهایی را تولید می کند.

۱-۴-۱ هم ترازی کاراکتری

از آنجا که دو خروجی ASR می توانند طول های متفاوت و ساختار ناهمسان داشته باشند، نخستین گام، هم تراز کردن آن ها است تا امکان مقایسه حرف به حرف فراهم شود. برای این منظور از الگوریتم فاصله لونیشتاین (Levenshtein Distance) استفاده شده است که در این پروژه دو راهبرد متفاوت برای هم تراز پیاده سازی و بررسی شد:

۱. هم تراز با کاراکتر تهی ($\emptyset$): در این حالت، در موقعیت هایی که یک دنباله کاراکتری در یکی از دو خروجی اضافه یا کم دارد، از یک نماد تهی به عنوان پرکننده استفاده می شود تا طول دو دنباله برابر شود.
۲. هم تراز با جایگزینی کاراکتر دنباله طولانی تر: در این راهبرد، به جای استفاده از کاراکتر تهی، کاراکتر متفاوت در دنباله کوتاه تر با کاراکتر متناظر در دنباله طولانی تر جایگزین می شود. برای نمونه، اگر یکی از مدل ها کلمه «خواب» و دیگری «خاب» را تولید کرده باشد، در این راهبرد به جای درج نماد $\emptyset$ برای هم تراز، کاراکتر متفاوت در دنباله کوتاه تر با کاراکتر متناظر در دنباله طولانی تر جایگزین می شود تا هر دو دنباله به شکل «خواب» درآیند و کاراکترهای متناظر در دو نما کاملاً منطبق شوند.

۲-۴-۱ سطح پردازش: کاراکتر در برابر زیرواژه

در این پروژه، واحد پردازش در دو حالت مختلف آزمایش شد:

- سطح کاراکتر: مدل در این حالت هر کاراکتر را به عنوان یک واحد مستقل در نظر می گیرد.
- سطح زیرواژه (Subword): در این حالت از قطعه بندی زیرواژه ای (مانند BPE یا SentencePiece) استفاده می شود.

سطح زیرواژه این مزیت را داشت که گاهی زیرواژه های اشتباه تولید شده، به دلیل حضور نداشتن در واژگان (دیکشنری) مدل، ناگزیر به شکل صحیح تری بازنویسی می شدند و بدین ترتیب برخی خطاها خود به خود اصلاح

می‌شدند. با این حال، در تنظیمات یکسان با مدل کاراکتری، نرخ خطای کاراکتر (CER) و نرخ خطای کلمه (WER) در مدل زیرواژه‌ای بالاتر بود؛ زیرا در این حالت، تولید نادرست یک زیرواژه واحد می‌تواند به‌طور نامتناسب، مقدار CER را افزایش دهد. به همین دلیل، در نهایت معماری اصلی پروژه بر پایهٔ سطح کاراکتر بنا شد.

انتخاب سطح کاراکتر به‌عنوان واحد پردازش، آگاهانه و با توجه به ویژگی‌های خط فارسی صورت گرفته است؛ زیرا بسیاری از خطاهای ASR روی فارسی، خطاهایی موضعی و نویسه‌ای هستند (مانند خطاهای هم‌آوایی یا فاصله‌گذاری) که در سطح کلمه به‌سختی قابل مدل‌سازی‌اند.

۱-۴-۳ رمزگذار و ادغام گیت‌دار و رمزگشا

پس از هم‌ترازی، بردارهای تعبیه‌شدهٔ دو نما به یک رمزگذار BiLSTM مشترک داده می‌شوند. سپس یک بلوک ادغام گیت‌دار (Gated Fusion) با گیت‌های متقاطع (Cross-view Gates) در هر گام زمانی به‌صورت پویا وزن هر نما را تعیین می‌کند. این مکانیزم به مدل امکان می‌دهد در هر موقعیت کاراکتری، به نمایی که شواهد قوی‌تری برای شکل صحیح ارائه می‌دهد، وزن بیشتری اختصاص دهد. در نهایت، یک رمزگشای LSTM یک‌جهته مجهز به مکانیزم توجه (Attention) از نوع Bahdanau، متن نهایی متناظر با فایل صوتی را تولید می‌کند.

۱-۵ ساختار گزارش

ادامهٔ این گزارش به‌شرح زیر سازمان‌دهی شده است. در فصل ۲، مروری بر کارهای مرتبط در حوزهٔ تصحیح پس از ASR و تصحیح خطای متون فارسی انجام می‌شود. فصل ۳ به‌تفصیل معماری پیشنهادی، شامل هم‌ترازی ورودی، رمزگذار BiLSTM، مکانیزم ادغام گیت‌دار، و رمزگشا را شرح می‌دهد. فصل ۴ به معرفی مجموعه داده، تنظیمات آموزش، و مدل‌های مقایسه اختصاص دارد. فصل ۵ نتایج تجربی و تحلیل خطاها را ارائه می‌کند و در نهایت فصل ۶ به جمع‌بندی و پیشنهاد کارهای آینده می‌پردازد.

فصل دوم

کارهای مرتبط

۱-۲ مقدمه

در سال‌های اخیر، با گسترش مدل‌های یادگیری عمیق و به‌ویژه معماری‌های مبتنی بر ترنسفورمر، حوزه تشخیص گفتار خودکار (ASR) پیشرفت چشمگیری داشته است. با این حال، خروجی این مدل‌ها همچنان در شرایط غیرایده‌آل و برای زبان‌های کم‌منبع حاوی خطاهای قابل‌توجهی است. به همین دلیل، رویکرد «تصحیح پس از تشخیص گفتار» (Post-ASR Correction) به‌عنوان یک لایهٔ پس‌پردازش مکمل، توجه بسیاری از پژوهشگران را به خود جلب کرده است. در این فصل، ابتدا مروری کلی بر روش‌های تصحیح پس از ASR ارائه می‌شود، سپس کارهای انجام‌شده به‌طور خاص برای زبان فارسی بررسی می‌گردد. در ادامه، رویکردهای ترکیب چندنمایی و سیستم‌های ترکیبی معرفی می‌شوند و در نهایت، استفاده از مدل‌های زبانی بزرگ در این حوزه مرور می‌شود.

۲-۲ تصحیح پس از تشخیص گفتار: رویکردهای عمومی

تصحیح پس از ASR به مجموعه روش‌هایی گفته می‌شود که پس از تولید متن اولیه توسط سیستم بازشناسی گفتار، آن را با هدف کاهش خطاهای جانشینی، درج، حذف و مرز کلمه اصلاح می‌کنند. این روش‌ها را می‌توان

به چند دسته اصلی تقسیم کرد:

روش‌های مبتنی بر مدل‌های دنباله‌به‌دنباله (Seq2Seq): در این رویکرد، یک مدل رمزگذار-رمزگشا آموزش می‌بیند تا خروجی خام ASR را به‌عنوان ورودی دریافت کرده و متن تصحیح‌شده را تولید کند. لی و همکاران (۲۰۲۴) یک مدل زبانی بزرگ چندزبانه را برای تصحیح پس از ASR روی چندین زبان از جمله هندی و نیپالی (هر دو با خط دواناگری) تنظیم دقیق کردند و از مزایای انتقال سطح-خط برای زبان‌های کم‌منبع بهره بردند. دوتا و همکاران (۲۰۲۲) مدل ROBERTA را معرفی کردند که یک مدل BART تنظیم‌شده برای تصحیح خطای ASR در زبان انگلیسی با تمرکز بر گفتار لهجه‌دار است و از توالی واج‌ها به‌عنوان ورودی اضافی بهره می‌برد.

روش‌های مبتنی بر تشخیص موجودیت‌های نادرست: شن و همکاران (۲۰۲۲) استراتژی MaskCorrect را معرفی کردند که در آن توکن‌های صحیح ماسک می‌شوند تا از کپی‌برداری ساده توسط مدل‌های ترنسفورمر جلوگیری شود.

روش‌های ترکیبی مبتنی بر رأی‌گیری: روش کلاسیک ROVER که توسط فیسکوس (۱۹۹۷) معرفی شد، خروجی چند سیستم بازشناسی را در سطح کلمه هم‌تراز کرده و با رأی‌گیری، خروجی نهایی را تولید می‌کند. این روش اگرچه در زبان‌های پرمنبع مؤثر است، اما در زبان‌های کم‌منبع و به‌ویژه برای خط‌هایی مانند فارسی که خط‌های موضعی و نویسه‌ای در آن‌ها نقش پررنگی دارد، ممکن است کارایی محدودی داشته باشد.

۲-۳ تصحیح پس از ASR برای زبان فارسی

زبان فارسی به‌دلیل ویژگی‌های خاص خط و ساختار زبانی خود، چالش‌های منحصربه‌فردی را در حوزه تصحیح پس از ASR ایجاد می‌کند. در سال‌های اخیر، چندین پژوهش به‌طور خاص به این موضوع پرداخته‌اند:

GEC-RAG (تصحیح مولد خطا با تولید افزوده‌شده با بازیابی): این روش که توسط پژوهشگران در سال ۲۰۲۵ معرفی شد، یک رویکرد مبتنی بر تولید افزوده‌شده با بازیابی (RAG) را برای تصحیح خطای ASR در زبان‌های کم‌منبع از جمله فارسی پیشنهاد می‌کند. در این روش، سیستم ASR به‌عنوان یک جعبه‌سیاه در نظر گرفته می‌شود و با ساخت یک پایگاه دانش که پیش‌بینی‌های ASR (بهترین فرضیه و ۵ فرضیه برتر) را با متن مرجع متناظر جفت می‌کند، نمونه‌های مشابه واژگانی را با معیار TF-IDF بازیابی کرده و همراه با رونوشت ASR به مدل زبانی مولد ارائه می‌دهد تا تصحیح هدفمند انجام شود. نتایج این پژوهش نشان داد که این استراتژی به‌طور قابل‌توجهی WER را در زبان فارسی کاهش می‌دهد و پتانسیل آن را برای انطباق دامنه و سناریوهای کم‌منبع برجسته می‌کند.

تصحیح خطای ASR با آگاهی از نویز (Noise-Aware): رحمانی و صمتی (۲۰۲۵) یک چارچوب تصحیح خطای ASR مقاوم به نویز را معرفی کردند که فرضیه‌های چندگانه و مدل‌سازی آگاه از نویز را ترکیب می‌کند. در این روش، از یک رمزگشای Whisper-large اصلاح‌شده برای تولید ۵ فرضیه برتر از گفتار نویزی فارسی استفاده می‌شود و سپس «تعبیه نویز سطح خطا» (Error Level Noise، ELN) به‌عنوان بازنمایی‌ای که عدم‌توافق معنایی و توکنی میان فرضیه‌ها را ثبت می‌کند، معرفی می‌گردد. مدل نهایی با تلفیق این تعبیه‌ها، WER را کاهش داد.

۲-۴ ترکیب چندنمایی و سیستم‌های ترکیبی

ایده استفاده از خروجی چندین سیستم بازشناسی و ترکیب آن‌ها برای کاهش خطا، سابقه‌ای طولانی در حوزه ASR دارد. روش ROVER که پیش‌تر اشاره شد، یکی از اولین و شناخته‌شده‌ترین رویکردها در این زمینه است. با این حال، این روش در سطح کلمه عمل می‌کند و ممکن است نتواند از قواعد ظریف سطح کاراکتر بهره‌برد. رویکرد چندنمایی در سطح کاراکتر: ریجوانی و همکاران (۲۰۲۰) یک مدل LSTM دو-رمزگذار چندنمایی را برای زبان‌های در خطر انقراض از جمله زبان یاکا (با خط دواناگری) پیشنهاد کردند که حدود ۳۴٪ کاهش خطا را به‌همراه داشت. این رویکرد نشان می‌دهد که ترکیب چند نمای مختلف از یک ورودی واحد می‌تواند به‌طور مؤثر خطاها را کاهش دهد.

مکانیزم ادغام گیت‌دار: در معماری‌های چندنمایی مدرن، مکانیزم‌های گیت‌دار (Gated Fusion) به‌عنوان راهکاری برای وزن‌دهی پویا به هر نما در هر گام زمانی مورد استفاده قرار می‌گیرند. این مکانیزم به مدل اجازه می‌دهد تا به‌جای وزن‌دهی ثابت به همه نماها، در هر موقعیت مکانی تصمیم بگیرد که به کدام نما بیشتر اتکا کند.

۲-۵ تصحیح خطای OCR و ارتباط آن با تصحیح ASR

با توجه به شباهت ساختاری میان وظیفه تصحیح خطای OCR و تصحیح پس از ASR (هر دو با ورودی نویزی متنی و هدف تولید متن پاک‌شده سروکار دارند)، مروری بر کارهای انجام‌شده در حوزه تصحیح OCR فارسی می‌تواند مفید باشد.

روش‌های قاعده‌مبنا: خسرویگی و همکاران (۲۰۲۰) یک رویکرد پس‌پردازش قاعده‌مبنا برای بهبود عملکرد OCR فارسی پیشنهاد کردند که بر حذف حروف بی‌معنا و دستکاری حداقلی متن تمرکز دارد و نیازی به مجموعه داده بزرگ ندارد. سرخی و همکاران نیز یک سیستم خبره برای تصحیح خودکار خطاهای املایی زبان

فارسی ارائه کردند.

روش‌های مبتنی بر یادگیری عمیق: ما و همکاران (۲۰۲۲) یک روش مبتنی بر قاعده و یادگیری عمیق برای تصحیح خطاهای املایی در متون فارسی پیشنهاد کردند. همچنین، تحلیل مقایسه‌ای مدل‌های زبانی بزرگ برای پس‌پردازش OCR فارسی (از ParsBERT تا GPT) نشان داد که مدل‌های زبانی بزرگ می‌توانند به‌طور مؤثر خطاهای OCR فارسی را که ناشی از ویژگی‌های خاص خط فارسی هستند، کاهش دهند.

ارتباط با تصحیح ASR: تجربه‌های موفق در تصحیح OCR فارسی نشان می‌دهد که رویکردهای سطح کاراکتر و استفاده از ویژگی‌های خاص خط فارسی (مانند اعراب، همزه، و نویسه‌های چندشکلی) می‌تواند مؤثر باشد. این بینش، انگیزه اصلی انتخاب سطح کاراکتر به‌عنوان واحد پردازش در پروژه حاضر را تقویت می‌کند.

۲-۶ مدل‌های زبانی بزرگ در تصحیح پس از ASR

در سال‌های اخیر، مدل‌های زبانی بزرگ (LLM) به‌عنوان ابزاری قدرتمند برای تصحیح پس از ASR مورد توجه قرار گرفته‌اند. این مدل‌ها به‌دلیل توانایی درک زمینه و تولید متن روان، می‌توانند خطاهای ASR را به‌طور مؤثر تصحیح کنند.

رویکردهای Zero-shot: در حالت بدون آموزش (Zero-shot)، مدل‌های زبانی بزرگ تنها با استفاده از دستورات متنی (Prompt) و بدون هیچ‌گونه تنظیم دقیق، خطاهای ASR را تصحیح می‌کنند. اگرچه این رویکرد برای زبان‌های پرمبغ نتایج قابل قبولی به‌همراه داشته است، اما در زبان‌های کم‌مبغ مانند فارسی، عملکرد آن محدود است.

تنظیم دقیق (Fine-tuning): کومار و همکاران (۲۰۲۶) در یک مطالعه مقایسه‌ای بر روی زبان هندی، نشان دادند که مدل‌های کوچک‌تر و تنظیم‌شده (مانند mT5 و ByT5) به‌طور مداوم از مدل‌های زبانی بزرگ‌تر در هر دو حالت تنظیم دقیق و یادگیری درون‌متنی بهتر عمل می‌کنند. یافته‌های آن‌ها نشان داد که ByT5 برای تصحیح سطح کاراکتر (مانند transliteration و تقسیم‌بندی کلمات) مؤثرتر است، در حالی که mT5 ناسازگاری‌های معنایی گسترده‌تر را مدیریت می‌کند.

چالش‌های مدل‌های زبانی بزرگ: ژو و همکاران (۲۰۲۵) نشان دادند که مدل‌های زبانی بزرگ ممکن است در وظیفه تصحیح پس از ASR دچار «تصحیح بیش‌ازحد» (Over-correction) یا «بازنویسی» (Paraphrasing) شوند که به افزایش خطاها منجر می‌شود. این چالش به‌ویژه در زبان‌های کم‌مبغ که داده‌های آموزشی محدودی برای تنظیم دقیق وجود دارد، بیشتر نمود پیدا می‌کند.

مرور کارهای مرتبط نشان می‌دهد که اگرچه رویکردهای مختلفی برای تصحیح پس از ASR پیشنهاد شده است، اما برای زبان فارسی به‌طور خاص، پژوهش‌ها عمدتاً بر استفاده از مدل‌های زبانی بزرگ و رویکردهای مبتنی بر RAG متمرکز بوده‌اند و کمبود مطالعاتی که از ترکیب چندنمایی خروجی مدل‌های ASR ناهمگون با مکانیزم گیت‌دار در سطح کاراکتر بهره‌برند، محسوس است. پژوهش حاضر با الهام از موفقیت رویکردهای چندنمایی در تصحیح OCR و با بهره‌گیری از تجربهٔ مدل‌سازی سطح کاراکتر، گامی در جهت پر کردن این خلأ برداشته و یک چارچوب تصحیح پس از ASR مبتنی بر ترکیب چندنمایی را برای زبان فارسی بررسی می‌کند.

فصل سوم

روش شناسی

۳-۱ فرمول بندی مسئله

فرض کنید برای یک utterance صوتی، دو فرضیه مستقل از دو مدل ASR به دست آمده است: فرضیه نخست از Whisper به صورت

$$\mathbf{x}^{(1)} = [x_1^{(1)}, x_2^{(1)}, \dots, x_n^{(1)}]$$

و فرضیه دوم از MMS به صورت

$$\mathbf{x}^{(2)} = [x_1^{(2)}, x_2^{(2)}, \dots, x_m^{(2)}]$$

که در آن ها n و m طول دنباله های کارا کتری هر فرضیه هستند و می توانند با یکدیگر متفاوت باشند. هدف مدل تصحیح پس از ASR، یادگیری یک نگاشت از این دو ورودی نویزی به متن مرجع (Ground Truth) یعنی

$$\mathbf{y} = [y_1, y_2, \dots, y_K]$$

است. به بیان دیگر، مدل باید دنباله تصحیح شده $\hat{\mathbf{y}}$ را به گونه ای بیابد که احتمال شرطی زیر بیشینه شود:

$$\hat{\mathbf{y}} = \arg \max_{\mathbf{y}'} p(\mathbf{y}' | \mathbf{x}^{(1)}, \mathbf{x}^{(2)}) \quad (۳-۱)$$

۲-۳ معماری مدل‌های ASR پایه

پیش از معرفی معماری مدل پیشنهادی، لازم است دو مدل ASR که به عنوان منابع تولید نمای اولیه در این پروژه به کار رفته‌اند، به طور مختصر معرفی شوند. دلیل انتخاب این دو مدل، همان طور که در فصل های پیش اشاره شد، تفاوت در معماری و داده آموزشی آن‌ها است که به مکمل بودن الگوهای خطایشان می‌انجامد.

۱-۲-۳ Whisper مدل

Whisper یک مدل ASR چندزبانه است که توسط OpenAI معرفی شده و بر پایه معماری رمزگذار-رمزگشای ترنسفورمر (Encoder-Decoder Transformer) بنا شده است. این مدل با نظارت ضعیف (Weak Supervision) روی ۶۸۰ هزار ساعت داده صوتی چندزبانه که از منابع اینترنتی گردآوری شده، آموزش دیده است. ورودی این مدل، spectrogram لگاریتمی-مل از سیگنال صوتی است که پس از دو لایه کانولوشن، به رمزگذار ترنسفورمر وارد می‌شود. رمزگشا نیز به صورت autoregressive توکن‌های متنی را تولید می‌کند. در این پروژه از نسخه Whisper-large-v۳ استفاده شده که بزرگ‌ترین نسخه موجود از این خانواده با حدود یک و نیم میلیارد پارامتر است.

۲-۲-۳ MMS مدل

MMS (Massively Multilingual Speech) توسط Meta معرفی شده و هدف آن پوشش بیش از ۱۰۰۰ زبان است. برخلاف Whisper که از معماری رمزگذار-رمزگشا استفاده می‌کند، MMS بر پایه معماری wav۲vec ۰.۲ طراحی شده است که یک مدل خودنظارتی (Self-Supervised) برای بازنمایی گفتار است. در این معماری، ابتدا یک رمزگذار کانولوشنی، بازنمایی‌های پنهان را از سیگنال صوتی خام استخراج می‌کند و سپس یک سر طبقه‌بندی CTC (Connectionist Temporal Classification) بر روی آن قرار می‌گیرد. در این پروژه از نسخه MMS-1B-All با حدود ۱ میلیارد پارامتر استفاده شده است.

تفاوت بنیادین این دو مدل از نظر معماری (ترنسفورمر در برابر wav۲vec ۰.۲) و از نظر داده آموزشی (۶۸۰ هزار ساعت داده متنوع در برابر ۱۱۰۰ زبان با داده کمتر برای هر زبان) باعث می‌شود که الگوهای خطای آن‌ها در مواجهه با فارسی متفاوت و در بسیاری از موارد مکمل یکدیگر باشد.

۳-۳ پیش‌پردازش و هم‌ترازی ورودی

از آنجا که دو فرضیه $x^{(1)}$ و $x^{(2)}$ ممکن است طول‌های متفاوت داشته باشند و ساختار کاراکتری آن‌ها همسان نباشد، نخستین گام در مدل پیشنهادی، هم‌ترازی این دو دنباله است. برای این کار از الگوریتم برنامه‌ریزی پویا

(Dynamic Programming) بر پایه فاصله لَوْنشتاین (Levenshtein Distance) استفاده شده است که

کمترین تعداد عملیات درج، حذف و جایگزینی را برای تبدیل یک دنباله به دنباله دیگر می‌یابد.

در این پروژه، دو راهبرد متفاوت برای هم‌ترازی پیاده‌سازی و ارزیابی شد:

۳-۳-۱ راهبرد نخست — هم‌ترازی با کاراکتر تهی

در این حالت، در موقعیت‌هایی که یک کاراکتر در یکی از دنباله‌ها وجود دارد و در دنباله دیگر متناظر ندارد، یک نماد تهی ($\emptyset$) در محل خالی درج می‌شود. به این ترتیب، طول دو دنباله برابر شده و به یک نگاشت یک‌به‌یک بین کاراکترها می‌رسیم:

$$(\tilde{\mathbf{x}}^{(1)}, \tilde{\mathbf{x}}^{(2)}) = \text{Align}(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}) \in (\Sigma \cup \{\emptyset\})^T \times (\Sigma \cup \{\emptyset\})^T \quad (2-3)$$

که در آن Σ واژگان کاراکتری است و T طول نهایی دو دنباله هم‌تراز شده است.

۳-۳-۲ راهبرد دوم — هم‌ترازی با استفاده از کاراکتر دنباله طولانی‌تر

در این راهبرد، به جای درج نماد تهی در دنباله کوتاه‌تر، از کاراکتر متناظر در دنباله طولانی‌تر استفاده می‌کنیم و در واقع کاراکتر اضافی موجود در دنباله طولانی‌تر را به هر دو دنباله اعمال می‌کنیم. برای نمونه، کلمه «خواب» یک کاراکتر بیشتر از «خاب» دارد. در این راهبرد، به جای اینکه در دنباله کوتاه‌تر («خاب») در موقعیت کاراکتر «و» یک نماد تهی درج شود تا دو دنباله هم‌طول شوند، این جای خالی با کاراکتر «و» از دنباله طولانی‌تر پر می‌شود؛ در نتیجه، هر دو دنباله به شکل «خواب» درمی‌آیند و کاراکترهای متناظر در دو نما کاملاً منطبق می‌شوند.

مزیت راهبرد دوم

این روش هم‌ترازی در کل به خروجی‌هایی منجر می‌شود که از دید یک ناظر انسانی، معنی‌دارتر و بهتر باشند؛ زیرا باعث می‌شود که برای مثال دو کلمه «خاب» و «خواب» هر دو به «خواب» تبدیل شوند و در نتیجه مدل پیشنهادی هم کلمه «خواب» را در خروجی کپی خواهد کرد که کلمه درستی است. در مقابل، اگر این دو دنباله با علامت تهی هم‌تراز شوند، دو مشکل بالقوه پیش می‌آید: نخست، مدل ممکن است کاراکتر تهی را با کاراکتر نامرتبط دیگری پر کند که به یک کلمه بی‌معنی یا نادرست از نظر نگارشی منجر شود؛ دوم، ممکن است مدل به جای بازسازی شکل صحیح، عیناً کلمه «خاب» را در خروجی کپی کند.

ایراد راهبرد دوم

با این حال، این روش یک ایراد مهم نیز دارد. گاهی ممکن است یکی از مدل‌های ASR کاراکترهای اشتباه اضافی در خروجی خود تولید کرده باشد که در خروجی مدل دیگر وجود ندارند. در این حالت، راهبرد دوم به جای

درج نماد تهی در رشته تولیدشده توسط ASR که کاراکتر اشتباه اضافی تولید نکرده است، همان کاراکترهای اشتباه را برای هم‌ترازی به خروجی درست نیز تحمیل می‌کند و در نتیجه هر دو خروجی ASR شامل کاراکترهای اشتباه اضافی می‌شوند. این موضوع به افزایش CER و WER در هنگام استفاده از این روش هم‌ترازی منجر می‌شود. به همین دلیل، در این پروژه راهبرد نخست (هم‌ترازی با کاراکتر تهی) به‌عنوان راهبرد پیش‌فرض انتخاب شد؛ زیرا در آن، اطلاعات تمایزی موجود در تفاوت دو نما حفظ می‌شود و مدل بهتر می‌تواند یاد بگیرد کدام نما در آن موقعیت قابل‌اعتمادتر است.

نکته مهم آن است که نماد $\emptyset$ در واژگان منبع به‌عنوان یک توکن عادی گنجانده می‌شود. در مرحله attention رمزگشا، تنها موقعیت‌های مربوط به padding دسته‌ای ماسک می‌شوند و نماد $\emptyset$ قابل‌مشاهده باقی می‌ماند تا مدل بتواند با استفاده از گیت‌ها و مکانیزم توجه تصمیم بگیرد که آیا یک شکاف هم‌ترازی باید نادیده گرفته شود یا خیر.

۳-۴ معماری مدل پیشنهادی

معماری مدل پیشنهادی از سه مؤلفه اصلی تشکیل شده است: رمزگذار مشترک BiLSTM، بلوک ادغام گیت‌دار، و رمزگشای LSTM مجهز به مکانیزم توجه.

۳-۴-۱ رمزگذار مشترک

در هر گام زمانی t ، ابتدا نمادهای هم‌ترازشده از دو خروجی ASR تعبیه (Embed) شده و سپس به‌صورت الحاقی به رمزگذار داده می‌شوند:

$$\mathbf{e}_t^{(1)} = \text{Emb}(\tilde{x}_t^{(1)}), \quad \mathbf{e}_t^{(2)} = \text{Emb}(\tilde{x}_t^{(2)}) \quad (۳-۳)$$

$$\mathbf{e}_t = [\mathbf{e}_t^{(1)}; \mathbf{e}_t^{(2)}] \in \mathbb{R}^{2E} \quad (۴-۳)$$

که در آن E بُعد تعبیه کاراکتری است. سپس یک BiLSTM بر روی دنباله $\mathbf{e}_{1:T}$ اعمال می‌شود و بازنمایی‌های پنهان را تولید می‌کند:

$$\mathbf{H} = \mathbf{h}_{1:T} = \text{BiLSTM}(\mathbf{e}_{1:T}) \quad (۵-۳)$$

که در آن $\mathbf{h}_t \in \mathbb{R}^{2H}$ و H بُعد پنهان هر جهت LSTM است.

BiLSTM چیست و چرا از آن استفاده شد؟

LSTM (Long Short-Term Memory) یک نوع شبکه عصبی بازگشتی (RNN) است که برای حل مشکل محو یا انفجار گرادین در RNN های ساده طراحی شده است. در یک RNN ساده، اطلاعات به صورت زنجیره ای از یک گام زمانی به گام بعدی منتقل می شود و به همین دلیل، شبکه در به خاطر سپردن وابستگی های بلندمدت دچار ضعف می شود. LSTM این مشکل را با معرفی یک سلول حافظه داخلی و سه گیت حل می کند:

- گیت ورودی (Input Gate): تعیین می کند چه مقدار از اطلاعات جدید وارد سلول حافظه شود.
- گیت فراموشی (Forget Gate): تعیین می کند چه مقدار از اطلاعات قدیمی حافظه نادیده گرفته شود.
- گیت خروجی (Output Gate): تعیین می کند چه مقدار از اطلاعات حافظه به عنوان خروجی گام جاری منتشر شود.

BiLSTM (Bidirectional LSTM) از دو LSTM مستقل تشکیل شده است که یکی از آن ها دنباله ورودی را از چپ به راست (Forward) و دیگری از راست به چپ (Backward) پردازش می کند. در هر گام زمانی، خروجی نهایی BiLSTM از الحاق (Concatenation) خروجی این دو LSTM به دست می آید. بنابراین، بعد خروجی BiLSTM در هر گام، دو برابر بعد پنهان یک LSTM تک جهت است.

دلیل استفاده از BiLSTM در این پروژه: در وظیفه تصحیح متن، تعیین کاراکتر صحیح در هر موقعیت نیازمند در نظر گرفتن زمینه (Context) از هر دو جهت چپ و راست است.

این رمزگذار میان هر دو نما مشترک است؛ به این معنا که هر دو نما از طریق الحاق به یک BiLSTM واحد وارد می شوند. انتخاب رمزگذار مشترک، هم پارامترهای مدل را کاهش می دهد و هم اجازه می دهد که اطلاعات دو نما با یکدیگر تلفیق شوند.

۳-۴-۲ مکانیزم ادغام گیت دار

هسته اصلی معماری پیشنهادی، بلوک ادغام گیت دار است که میان رمزگذار و رمزگشا قرار می گیرد و وظیفه آن وزن دهی پویا به دو نما در هر گام زمانی است.

فرمول اصلی مقاله

در مقاله مرجع، ابتدا دو جریان $v_t^{(1)}$ و $v_t^{(2)}$ با استفاده از لایه های خطی از بردار پنهان مشترک h_t که خروجی رمزگذار است، استخراج می شوند و سپس گیت های متقاطع به صورت زیر محاسبه می شوند:

$$g_{12,t} = \sigma \left(W_{12} [v_t^{(1)}; v_t^{(2)}] \right) \in \mathbb{R}^H \quad (۳-۶)$$

$$\mathbf{g}_{21,t} = \sigma\left(W_{21}[\mathbf{v}_t^{(2)}; \mathbf{v}_t^{(1)}]\right) \in \mathbb{R}^H \quad (۷-۳)$$

و بازنمایی‌های تعدیل‌شده به‌صورت زیر محاسبه می‌شوند:

$$\hat{\mathbf{h}}_t^{(1)} = \mathbf{v}_t^{(1)} + \mathbf{g}_{12,t} \odot \tanh(\mathbf{v}_t^{(2)}) \quad (۸-۳)$$

$$\hat{\mathbf{h}}_t^{(2)} = \mathbf{v}_t^{(2)} + \mathbf{g}_{21,t} \odot \tanh(\mathbf{v}_t^{(1)}) \quad (۹-۳)$$

$$\hat{\mathbf{h}}_t = [\hat{\mathbf{h}}_t^{(1)}; \hat{\mathbf{h}}_t^{(2)}] \in \mathbb{R}^{2H} \quad (۱۰-۳)$$

در این فرمول، هر گیت نقشی به‌عنوان یک «تعدیل‌کننده باقی‌مانده» (Residual Modulator) ایفا می‌کند: اطلاعات نمای مقابل، پس از عبور از تابع $\tanh$ و ضرب در گیت، به‌صورت یک جمله افزودنی روی نمای اصلی سوار می‌شود.

فرمول پیاده‌سازی‌شده در این پروژه

در پیاده‌سازی این پروژه، فرمول‌ها به‌صورت زیر بازنویسی شدند:

$$\mathbf{v}_t^{(1)} = \text{LN}(W_1 \mathbf{h}_t + b_1), \quad \mathbf{v}_t^{(2)} = \text{LN}(W_2 \mathbf{h}_t + b_2) \quad (۱۱-۳)$$

$$\mathbf{g}_{12,t} = \sigma\left(W_{12}[\mathbf{v}_t^{(1)}; \mathbf{v}_t^{(2)}] + b_{12}\right) \quad (۱۲-۳)$$

$$\mathbf{g}_{21,t} = \sigma\left(W_{21}[\mathbf{v}_t^{(2)}; \mathbf{v}_t^{(1)}] + b_{21}\right) \quad (۱۳-۳)$$

$$\hat{\mathbf{h}}_t^{(1)} = (1 - \mathbf{g}_{12,t}) \odot \mathbf{v}_t^{(1)} + \mathbf{g}_{12,t} \odot \mathbf{v}_t^{(2)} \quad (۱۴-۳)$$

$$\hat{\mathbf{h}}_t^{(2)} = \mathbf{g}_{21,t} \odot \mathbf{v}_t^{(1)} + (1 - \mathbf{g}_{21,t}) \odot \mathbf{v}_t^{(2)} \quad (۱۵-۳)$$

$$\hat{\mathbf{h}}_t = [\hat{\mathbf{h}}_t^{(1)}; \hat{\mathbf{h}}_t^{(2)}] \in \mathbb{R}^{2H} \quad (۱۶-۳)$$

که در آن $\text{LN}(\cdot)$ عملگر Layer Normalization، σ تابع سیگموئید و $\odot$ ضرب عنصر به‌عنصر است.

مقایسه دو فرمول

تفاوت اول — افزودن Layer Normalization پیش از محاسبه گیت‌ها:

در فرمول اصلی مقاله، دو جریان $\mathbf{v}_t^{(1)}$ و $\mathbf{v}_t^{(2)}$ به‌صورت مستقیم از لایه‌های خطی به‌دست می‌آیند و مقیاس آن‌ها می‌تواند بسیار متفاوت باشد. اما از آنجا که گیت‌ها خروجی تابع سیگموئید هستند و در بازه $(0, 1)$ قرار دارند، و تابع $\tanh$ نیز خروجی خود را در بازه $(-1, 1)$ محدود می‌کند، جمله $\mathbf{g}_{12,t} \odot \tanh(\mathbf{v}_t^{(2)})$ حداکثر در بازه تقریبی $(-1, 1)$ قرار می‌گیرد.

اکنون اگر مقیاس $v_t^{(1)}$ به طور قابل توجهی بزرگتر از ۱ باشد (مثلاً در بازه $[-10, 10]$)، آنگاه افزودن جمله‌ای که قدر مطلق آن حداکثر ۱ است، اثر نسبی بسیار ناچیزی بر خروجی $\hat{h}_t^{(1)}$ خواهد داشت. در چنین شرایطی، مکانیزم گیت عملاً نمی‌تواند نمای اصلی را به‌سوی نمای مقابل تعدیل کند و هدف اصلی طراحی گیت‌های متقاطع — که همان انتخاب پویا و مؤثر میان دو نما است — محقق نمی‌شود.

افزودن Layer Normalization پس از لایه‌های خطی، دو جریان $v_t^{(1)}$ و $v_t^{(2)}$ را به یک مقیاس مشترک نرمال‌سازی می‌کند و تضمین می‌کند که جمله گیت‌دار از نظر مقیاس با نمای اصلی قابل مقایسه باشد. در نتیجه، گیت‌ها می‌توانند بدون توجه به تفاوت‌های مقیاس، بر اساس محتوای معنایی دو نما تصمیم بگیرند و تأثیر تعدیل‌کننده آن‌ها واقعاً بر خروجی نهایی ظاهر شود.

تفاوت دوم — جایگزینی اتصال باقی‌مانده با ترکیب محدب:

در فرمول اصلی، $\hat{h}_t^{(1)} = v_t^{(1)} + g_{12,t} \odot \tanh(v_t^{(2)})$ است. این ساختار دو محدودیت دارد:

اول اینکه خروجی می‌تواند به‌طور نامحدود بزرگ شود، چون اطلاعات نمای مقابل به‌صورت جمعی اضافه می‌شود. اگر مقیاس $v_t^{(2)}$ بزرگ باشد، جمله گیت‌دار می‌تواند در جهت افزایش بی‌مهار خروجی عمل کند و پایداری عددی آموزش را مختل کند.

دوم اینکه نمای اصلی همیشه در خروجی حضور دارد و گیت تنها می‌تواند اطلاعات نمای مقابل را به آن اضافه کند. این یعنی حتی اگر نمای اصلی در یک موقعیت خاص کاملاً نادرست باشد، باز هم سهم آن در خروجی نهایی غیرصفر است و مدل نمی‌تواند آن را به‌طور کامل نادیده بگیرد. در نتیجه، مکانیزم گیت توانایی «انتخاب نرم» (Soft Selection) بین دو نما را به‌طور کامل ندارد.

در فرمول پیاده‌سازی‌شده، جایگزینی اتصال باقی‌مانده با ترکیب محدب، دو مزیت به‌همراه دارد:

- خروجی همواره در بازه مقادیر دو نما باقی می‌ماند و از کنترل خارج نمی‌شود؛ به این ترتیب پایداری عددی در طول آموزش تضمین می‌شود.

- گیت‌ها نقش یک «کلید انتخاب نرم» را ایفا می‌کنند. وقتی $g_{12,t} \rightarrow 1$ ، مدل به‌طور کامل به نمای دوم اتکا می‌کند و وقتی $g_{12,t} \rightarrow 0$ ، به نمای اول. این تفسیرپذیری بسیار بالاتری نسبت به فرمول اصلی فراهم می‌کند.

لازم به ذکر است که در فرمول پیاده‌سازی‌شده، به‌دلیل استفاده از ترکیب محدب، مشکل مقیاس موجود در فرمول اصلی به‌طور طبیعی وجود ندارد (چون $v_t^{(1)}$ و $v_t^{(2)}$ مستقیماً با هم ترکیب می‌شوند و واسطه‌ای به شکل $\tanh$ در میان نیست). بنابراین، استفاده از Layer Normalization در فرمول پیاده‌سازی‌شده بیشتر

به عنوان یک عامل افزایش دنده پایداری آموزش عمل می کند تا یک ضرورت ریاضی. با این حال، همین Layer Normalization به طور تجربی به کاهش حساسیت مدل به مقدار اولیه learning rate و هموارتر شدن منحنی loss کمک می کند.

۳-۴-۳ رمزگشا با مکانیزم توجه

رمزگشا یک LSTM یک جهته است که در هر گام زمانی k ، با استفاده از مکانیزم توجه جمعی (Additive Attention) از نوع Bahdanau، یک بردار زمینه (Context Vector) $\mathbf{c}_k$ را بر روی دنباله بازنمایی های ادغام شده $\hat{\mathbf{h}}_{1:T}$ محاسبه می کند. سپس کاراکتر بعدی $\hat{y}_k$ از ترکیب وضعیت رمزگشا و بردار زمینه پیش بینی می شود:

$$\mathbf{c}_k = \sum_{t=1}^T \alpha_{k,t} \hat{\mathbf{h}}_t, \quad \alpha_{k,t} = \frac{\exp(e_{k,t})}{\sum_{t'=1}^T \exp(e_{k,t'})} \quad (۱۷-۳)$$

$$\hat{y}_k \sim \text{Softmax}(W_o[\mathbf{s}_k; \mathbf{c}_k] + b_o) \quad (۱۸-۳)$$

که در آن $\mathbf{s}_k$ وضعیت پنهان رمزگشا در گام k است و $e_{k,t}$ امتیاز هم تراز می بین وضعیت رمزگشا و بازنمایی رمزگذار در موقعیت t محاسبه شده توسط شبکه توجه است.

آموزش مدل با کمینه سازی آنتروپی متقابل در سطح توکن و با استفاده از تکنیک teacher forcing انجام می شود و در زمان استنتاج، رمزگشایی حریصانه (Greedy Decoding) به کار می رود. انتخاب مدل نهایی بر اساس کمترین CER روی مجموعه اعتبارسنجی انجام می شود.

فصل چهارم

داده‌ها، نرمال‌سازی و پیاده‌سازی

پس از تبیین مبانی نظری و معماری مدل پیشنهادی در فصل پیشین، این فصل به تفصیل به فرآیند عملی اجرای پروژه اختصاص دارد. در ابتدا، مجموعه داده مورد استفاده و روند ساخت آن معرفی می‌شود. سپس، مراحل نرمال‌سازی متن و پالایش نمونه‌های نامعتبر تشریح می‌گردد. در ادامه، تنظیمات آموزش مدل و جزئیات پیاده‌سازی، شامل توجیه انتخاب هایپرپارامترها و تکنیک‌های منظم‌سازی، بیان می‌شود. در پایان، تحلیل جامعی از ماهیت نویز در خروجی مدل‌های ASR بر زبان فارسی ارائه می‌گردد و نتایج با مقاله مرجع مقایسه می‌شود.

۴-۱ مجموعه داده

۴-۱-۱ معرفی مجموعه داده اولیه

در این پروژه، از زیرمجموعه‌ای از مجموعه داده Common Voice نسخه ۲۲ برای زبان فارسی استفاده شده است. این زیرمجموعه که با نام `aliyazd95/common_voice_۲۲_۰_fa` در پلتفرم Hugging Face در دسترس قرار دارد، شامل نمونه‌های صوتی گویشوران فارسی به همراه متن مرجع متناظر با هر نمونه است. مجموعه داده Common Voice توسط بنیاد Mozilla گردآوری شده و یکی از بزرگ‌ترین مجموعه داده‌های صوتی متن‌باز برای زبان‌های مختلف جهان به‌شمار می‌رود.

۴-۱-۲ ساختار ستون‌های مجموعه داده

مجموعه داده مورد استفاده شامل ستون‌های زیر است:

- شناسه گویشور (client_id): شناسه یکتای گویشور برای تفکیک گویشوران.
- مسیر فایل (path): مسیر فایل صوتی در مخزن داده.
- داده صوتی (audio): داده صوتی خام که نرخ نمونه برداری تمام نمونه‌ها به ۱۶ کیلوهرتز تبدیل شده است.
- متن مرجع (sentence): متن مرجع متناظر با فایل صوتی.
- رأی‌های مثبت (up_votes): تعداد رأی‌های مثبت به کیفیت نمونه.
- رأی‌های منفی (down_votes): تعداد رأی‌های منفی به کیفیت نمونه.
- گروه سنی (age): گروه سنی گویشور.
- جنسیت (gender): جنسیت گویشور.
- لهجه (accent): لهجه گویشور.
- زبان و منطقه (locale): زبان و منطقه جغرافیایی؛ برای فارسی مقدار fa.
- بخش‌بندی (segment): اطلاعات بخش‌بندی نمونه.
- گونه زبانی (variant): نوع گونه زبانی.

۴-۱-۳ تفکیک داده‌ها

مجموعه داده از پیش به سه بخش آموزش (train)، اعتبارسنجی (validation) و آزمون (test) تفکیک شده است. این تفکیک توسط سازندگان مجموعه داده و به صورت مستقل از گویشور (Speaker-Independent) انجام شده است؛ به این معنا که گویشوران حاضر در مجموعه آزمون با گویشوران مجموعه آموزش مشترک نیستند و در نتیجه، مشکل نشت داده (Data Leakage) وجود ندارد. بنابراین، در این پروژه نیازی به تفکیک دستی داده‌ها نبود و از همان تقسیم‌بندی از پیش تعیین شده استفاده گردید. تعداد نمونه‌های اولیه در هر بخش در جدول ۴-۱ آمده است.

۴-۱-۴ تولید خروجی‌های ASR

نخستین گام عملی در این پروژه، تولید خروجی‌های اولیه دو مدل ASR بر روی نمونه‌های صوتی است. از آنجا که هدف اصلی این پروژه، تصحیح پس از تشخیص گفتار بر پایه ترکیب چندنمایی است، هر نمونه صوتی به‌طور جداگانه به دو مدل زیر داده شد:

۱. Whisper-large-v۳ به‌عنوان نمای اول، با تنظیم زبان به فارسی و وظیفه رونویسی.

۲. MMS-۱B-All به‌عنوان نمای دوم، با تنظیم زبان هدف به کد fas (کد زبان فارسی).

پردازش نمونه‌های صوتی به‌صورت دسته‌ای با اندازه ۸ نمونه در هر دسته انجام شد تا هم از حافظه GPU به‌طور بهینه استفاده شود و هم سرعت استنتاج افزایش یابد. برای تسریع استنتاج، از دقت نیمه‌دقیق (float۱۶) در صورت وجود GPU استفاده می‌شود. خروجی‌های هر دو مدل به‌همراه متن مرجع و شناسه نمونه، در یک فایل CSV ذخیره گردید.

برای اطمینان از پایداری فرآیند در برابر قطع شدن اجرا، مکانیزم نقطه بازرسی (checkpoint) در فواصل هر ۱۰۰ نمونه پیاده‌سازی شد؛ به این معنا که در صورت قطع شدن پردازش، می‌توان آن را از آخرین نمونه ذخیره‌شده ادامه داد.

۴-۲ نرمال‌سازی متن

۴-۲-۱ ضرورت نرمال‌سازی

داده‌های خام ASR و متن‌های مرجع، حاوی تنوع نوشتاری هستند که بدون نرمال‌سازی، مدل را در یادگیری الگوهای صحیح دچار مشکل می‌کنند. این تفاوت‌ها در پنج دسته اصلی قرار می‌گیرند:

نخست، نویسه‌های عربی در برابر فارسی. در متن‌های فارسی، ممکن است از نویسه‌های عربی مانند «ی» و «ك» به‌جای «ی» و «ک» فارسی استفاده شود.

دوم، نیم‌فاصله (ZWNJ). در خط فارسی، نیم‌فاصله (با کد یونیکد U+۲۰۰C) نقش مهمی در اتصال یا جداسازی اجزای کلمات دارد. نبود یا اشتباه بودن نیم‌فاصله (مانند «میروم» به‌جای «می‌روم») می‌تواند به خطاهای جانشینی و مرز کلمه منجر شود.

سوم، فاصله‌گذاری. فاصله‌های اضافی یا نادرست، به‌ویژه در ترکیبات و کلمات مرکب، از مشکلات رایج در خروجی‌های ASR است.

چهارم، اعراب‌گذاری و همزه. وجود یا نبود اعراب و همزه، تنوع‌های نوشتاری متعددی ایجاد می‌کند. پنجم، علائم نگارشی. یکی از مشکلات اساسی در مجموعه داده، ناهمسانی در استفاده از علائم نگارشی است.

برخی از برجسبها دارای علائم نگارشی کامل بودند (مانند نقطه، ویرگول، علامت سؤال) در حالی که برخی دیگر فاقد این علائم بودند. همچنین، مدل Whisper در خروجی خود علائم نگارشی را قرار می‌دهد، در حالی که MMS فاقد علائم نگارشی است. این ناهمسانی نویزی ایجاد می‌کند که یادگیری مدل را دشوار می‌سازد؛ زیرا مدل نمی‌تواند الگوی درستی از حضور یا عدم حضور علائم را بیاموزد. به همین دلیل، تمام علائم نگارشی از هر سه ستون (متن مرجع، خروجی Whisper و خروجی MMS) حذف شدند.

۲-۲-۴ کتابخانه Shekar

برای نرمال‌سازی متون فارسی، از کتابخانه Shekar استفاده شده است. این کتابخانه مجموعه‌ای از ابزارهای پیش‌پردازش زبان فارسی را در اختیار می‌گذارد که در جدول ۲-۴ معرفی شده‌اند. این کتابخانه به‌صورت زنجیره‌ای (Pipeline) عمل می‌کند؛ به این معنا که هر مؤلفه به‌ترتیب روی متن اعمال می‌شود و خروجی هر مرحله به‌عنوان ورودی مرحله بعدی استفاده می‌شود.

۳-۲-۴ مراحل نرمال‌سازی با مثال

نرمال‌سازی متن در سه ستون انجام شد: متن مرجع، خروجی Whisper و خروجی MMS. برای هر سه ستون، یک تابع یکسان اعمال شد.

نرمال‌سازی حروف و نویسه‌های یونیکد: تبدیل نویسه‌های عربی به معادل‌های فارسی. برای نمونه:

• «کتابی» ← «کتابی»

• «فاطمه» ← «فاطمه»

همچنین کشیدگی‌ها (کاراکتر «-») حذف می‌شوند.

نرمال‌سازی ارقام: ارقام عربی یا انگلیسی به فارسی تبدیل می‌شوند: «۰۱۲۳۴۵۶۷۸۹» ← «۰۱۲۳۴۵۶۷۸۹». حذف علائم نگارشی: علائمی مانند نقطه، ویرگول، علامت سؤال، علامت تعجب، دو نقطه، نقطه‌ویرگول، پرانتز و گیومه حذف می‌شوند.

حذف اعراب: اعراب‌های فتحه، ضمه، کسره، سکون، تشدید و تنوین حذف می‌شوند.

یکسان‌سازی فاصله‌ها: فاصله‌های اضافی حذف و فاصله‌های نادرست اصلاح می‌شوند.

حذف تکرار حروف: اگر حرفی به‌طور غیرطبیعی تکرار شده باشد، به شکل اصلی بازگردانده می‌شود: «مهمهمان»

← «مهمان».

۴-۲-۴ اصلاح خط الرسم اشتباه در مجموعه داده Common Voice

یکی از مشکلات اساسی مجموعه داده Common Voice، خط الرسم اشتباه در برخی از برچسب‌ها است. برای نمونه، کلماتی مانند «خانهای» به جای «خانه‌ای»، «صرفه‌جویی» به جای «صرفه‌جویی» و «بهموقع» به جای «به‌موقع» نوشته شده بودند.

انتخاب رویکرد اصلاح

برای اصلاح این مشکل، دو رویکرد ممکن بود:

رویکرد نخست — استفاده از مدل‌های زبانی بزرگ: می‌توان از LLM‌هایی مانند GPT برای اصلاح خودکار این خطاها استفاده کرد. با این حال، این رویکرد دو ایراد اساسی دارد: نخست، مدل‌های زبانی بزرگ ممکن است دچار توهم (Hallucination) شوند و عبارات را به‌طور ناخواسته بازنویسی یا پرفریز کنند؛ دوم، فرآیند اصلاح با LLM برای حجم بالای داده (بیش از ۳۰۰۰۰ نمونه) پرهزینه و زمان‌بر است.

رویکرد دوم — استفاده از قواعد رجکسی و لیست استثناها: در این پروژه، رویکرد دوم انتخاب شد. مجموعه‌ای از قواعد مبتنی بر عبارات باقاعده (Regular Expressions) طراحی شد که با شناسایی الگوهای اشتباه، آن‌ها را به شکل صحیح تبدیل می‌کردند. همچنین، یک لیست استثنا شامل کلماتی که ممکن بود به اشتباه اصلاح شوند، تهیه شد. این رویکرد از سه مزیت برخوردار است: قابل‌کنترل، سریع و بدون توهم است.

قواعد رجکسی به کاررفته

قواعد رجکسی متعددی برای اصلاح خط الرسم اشتباه طراحی و پیاده‌سازی شد. در ادامه، هر قاعده به‌طور دقیق توضیح داده می‌شود.

قاعده نخست — اصلاح پسوند «های/ها». این قاعده برای اصلاح کلماتی طراحی شده است که پسوند «های» یا «ها» به آن‌ها چسبیده است. الگوی این قاعده شامل پسوندهای «هایمان»، «هایتان»، «هایشان»، «هایم»، «هایت»، «هایش»، «هایی»، «های» و «ها» است. برای نمونه:

• «فعالیت‌های» ← «فعالیت‌های»

• «کتابهای» ← «کتاب‌های»

• «خانههای» ← «خانه‌های»

قاعده دوم — اصلاح «ه» چسبیده به ضمائر. این قاعده برای اصلاح کلماتی طراحی شده است که حرف «ه» به ضمائر متصل چسبیده است. ضمائر متصل در این الگو شامل «ام»، «ات»، «اش»، «ایم»، «اید»، «اند»،

«است»، «مان»، «تان»، «شان»، «ای»، «بود»، «شد»، «باش» و «شو» است. برای نمونه:

- «بودهام» ← «بودهام»
- «کردهاست» ← «کردهاست»
- «آمدهایم» ← «آمدهایم»
- «نکردهای» ← «نکردهای»

قاعده سوم — اصلاح «ه» چسبیده به «ی» اضافه. برای کلماتی که حرف «ه» به «ی» اضافه چسبیده است. برای نمونه: «خانهی» ← «خانه‌ی»، «مدرسه‌ی» ← «مدرسه‌ی»، «نکته‌ی» ← «نکته‌ی».

قاعده چهارم — اصلاح کلمات مرکب. برای کلماتی که اجزای آن‌ها بدون نیم‌فاصله نوشته شده‌اند. لیست کاملی از پسوندهای ترکیبی مانند پسوندهای زیر و مشتقات آنها در نظر گرفته شدند:

«گیر»، «گر»، «گزار»، «گذار»، «سازی»، «ساز»، «دار»، «بند»، «کش»، «آمیز»، «شناس»، «پذیر»، «ستان»، «پرور»، «باز»، «یاب»، «پیما»، «آور»، «بردار»، «سار»، «شهر»، «آباد»، «خانه»، «سرا»، «کده».

برای نمونه: «پرده‌برداری» ← «پرده‌برداری».

قاعده پنجم — اصلاح پسوندهای با «آ». برای کلماتی که پسوندی با حرف «آ» به کلمه قبلی چسبیده‌اند. پسوندهای با «آ» شامل «آمیز»، «آور»، «آباد»، «انداز»، «افزار»، «ابزار»، «انگیز»، «افزا»، «افروز»، «افکن»، «آلود»، «آسا»، «اندیش»، «آزما»، «آموز» و... است. برای نمونه: «دانش‌آموز» ← «دانش‌آموز»، «رعب‌انگیز» ← «رعب‌انگیز».

قاعده ششم — اصلاح «ی» چسبیده به پسوند. برای کلماتی که حرف «ی» به پسوند چسبیده است. پسوندهای این الگو شامل «ها»، «ام»، «ات»، «اش»، «ایم»، «اید»، «اند»، «است»، «مان»، «تان»، «شان»، «خواه»، «خواهی»، «خواهان» و «خواهانی» است. برای نمونه:

- «بیماریها» ← «بیماری‌ها»
- «بدبیارها» ← «بدبیار‌ها»
- «ایرانیام» ← «ایرانی‌ام»

قاعده هفتم — «به + کلمه». برای ترکیباتی که «به» به کلمه بعدی چسبیده است. دو گروه از کلمات وجود

دارد:

• گروه با نیم فاصله: «معنی»، «ویژه»، «جای»، «خاطر»، «نظر»، «طور»، «جز»، «مراتب»، «تدریج»، «سبب»، «ندرت»، «ناچار»، «اندازه»، «عنوان»، «خصوص»، «عمد»، «ظاهر»، «زحمت»، «اجبار»، «اختصار»، «اصطلاح»، «دلخواه»، «دنبال»، «عقبه»، «اثر»، «کلی»، «زودی»، «تفصیل»، «حالت»، «هنگام»، «مصاداق».

• گروه با فاصله: «سر»، «دست»، «طرف»، «راه»، «زور»، «کار»، «حال»، «دل»، «جان»، «سمت»، «فکر»، «چاه»، «کس»، «چیز»، «من»، «تو»، «او»، «ما»، «شما»، «آنها»، «ایشان»، «نام»، «دلیل»، «علت»، «طرز»، «محض»، «عقب»، «بالا»، «پایین»، «کنار»، «داخل»، «خارج»، «شکل»، «رو»، «زیر»، «موقع»، «پیگیری»، «خطر»، «استطاعت»، «هم»، «عهده»، «سوی»، «صورت»، «آسانی»، «سختی»، «منصه»، «آرامی»، «سرعت»، «راستی»، «وجود»، «دنیا»، «آهستگی»، «صرفه»، «وثیقه».

برای نمونه: «بهموقع» ← «به موقع»،

قاعده هشتم — «چه + کلمه». برای ترکیباتی که «چه» به کلمه بعدی چسبیده است. کلمات شامل «کس»، «کسی»، «چیز»، «وقت»، «موقع»، «زمان»، «زمانی»، «طور»، «گونه»، «قدر»، «اندازه»، «تعداد»، «نام»، «رنگ»، «کار»، «شغل»، «شهر»، «کشور»، «روز»، «ساعت»، «ساعتی»، «سال»، «ماه»، «فکری»، «حرفی»، «خبری»، «دلیلی»، «منظوری»، «حسی»، «نسبتی»، «شماره»، «قیمتی»، «مشکلی»، «اتفاقی»، «کلاسی»، «کتابی»، «درسی»، «غذایی»، «لباسی»، «مدلی»، «برنامه‌ای»، «راهی»، «نوع» است. برای نمونه: «چهکار» ← «چه کار»، «چهقدر» ← «چه قدر».

قاعده نهم — «همه + کلمه». کلمه «همه» را که به کلمه بعدی چسبیده است، با نیم فاصله جدا می‌کند: «همه‌کاره» ← «همه کاره».

قاعده دهم — «فی‌ال + کلمه». کلماتی مانند «فی‌المثل» را که «فی» به «ال» چسبیده است، اصلاح می‌کند: «فی‌المثل» ← «فی‌المثل».

قاعده یازدهم — «بین‌ال + کلمه». کلماتی مانند «بین‌المللی» را اصلاح می‌کند: «بین‌المللی» ← «بین‌المللی».

لیست استنها

لیست استنها شامل حدود چند صد کلمه است که نباید توسط قواعد Regex اصلاح شوند. نمونه‌هایی از این کلمات:

کلماتی که با «ه» تمام می‌شوند: «ماهی»، «شاهی»، «چاهی»، «راهی»، «سیاهی»، «کوتاهی»، «پناهی»، «گناهی»، «گروهی»، «روباهی»، «تنبیهی»، «سپاهی»، «گاهی»، «نگاهی»، «آهی»، «پادشاهی»، «کوهی»،

«دهی»، «آگاهی»، «تباهی»، «گیاهی»، «فکاهی»، «الهی»، «بدیهی»، «گواهی»، «خواهی»، «تنهایی»، «دستگاهی»، «گاهوار»، «شاهوار»، «منتهی»، «وجهی»، «کاهی»، «تهی»، «تهیدست»، «هیاو»، «اکراهی»، «ایستگاهی»، «خیرخواهی»، «بدخواهی»، «شاهکار»، «بدهکار»، «کوهسار»، «بزهکار»، «آهنگر»، «صبحگاهی»، «شامگاهی»، «شبانگاهی».

اهمیت این لیست را می‌توان با یک مثال نشان داد: بدون این لیست، قاعده «ه + اضافه» ممکن بود کلمه «گناهی» را به اشتباه به «گناهی» تبدیل کند، در حالی که شکل صحیح آن «گناهی» است.

استفاده از لیست سفید

یکی دیگر از مشکلات اساسی، کلماتی بود که با «ه + ای» تمام می‌شدند. اگر یک قاعده رجکسی نوشته می‌شد که هر «های» را به «های» تبدیل کند، این قاعده تمام کلماتی که پسوند «های» داشتند را نیز خراب می‌کرد (مثلاً «فعالتهای» را به «فعالتهای» تبدیل می‌کرد). به همین دلیل، یک لیست سفید (Whitelist) برای اصلاح این کلمات ساخته شد. نمونه‌هایی از این لیست سفید در جدول ۴-۳ آمده است.

۴-۳ پالایش داده‌های نامعتبر

۴-۳-۱ ضرورت پالایش

پس از تولید خروجی‌های ASR و نرمال‌سازی متون، برخی از نمونه‌ها به دلیل کیفیت پایین برای آموزش مدل مناسب نیستند:

- نمونه‌هایی که خروجی یکی از ASRها یا هر دو خالی است.
- نمونه‌هایی که نرخ خطای کاراکتری (CER) یا نرخ خطای کلمه‌ای (WER) در آن‌ها بسیار بالا است.
- نمونه‌هایی که متن مرجع آن‌ها بسیار کوتاه یا بسیار بلند است و نسبت طول خروجی ASR به متن مرجع خارج از محدوده معقول است.
- نمونه‌هایی که نسبت کاراکترهای فارسی در متن مرجع آن‌ها پایین است.

۴-۳-۲ نتایج پالایش

پس از اعمال معیارهای پالایش، تعداد نمونه‌های حذف‌شده در هر بخش به شرح جدول ۴-۴ است. دلایل عمده نامعتبر بودن نمونه‌ها شامل CER بالا در هر دو مدل (بیشترین دلیل)، WER بالا در هر دو مدل، خروجی خالی، نسبت طول نامناسب و نسبت کاراکترهای فارسی پایین بود.

جدول ۴-۱: تعداد نمونه‌های اولیه در هر بخش از مجموعه داده

بخش	تعداد نمونه اولیه
آموزش (train)	۲۹۷۸۹
اعتبارسنجی (validation)	۱۰۶۷۶
آزمون (test)	۱۰۶۷۶
مجموع	۵۱۱۴۱

جدول ۴-۲: مؤلفه‌های کتابخانه Shekar مورد استفاده

مؤلفه	وظیفه
AlphabetNormalizer	یکسان‌سازی حروف الفبا و تبدیل نویسه‌های عربی به فارسی
ArabicUnicodeNormalizer	نرمال‌سازی نویسه‌های یونیکد عربی
DigitNormalizer	یکسان‌سازی ارقام عربی و فارسی
PunctuationRemover	حذف علائم نگارشی
RemoveDiacritics	حذف اعراب‌گذاری
RepeatedLetterNormalizer	حذف تکرار حروف
SpacingNormalizer	یکسان‌سازی فاصله‌ها
YaNormalizer	یکسان‌سازی نویسه «ی»

جدول ۴-۳: نمونه‌هایی از لیست سفید برای اصلاح کلمات با «ای»

شکل اشتباه	شکل صحیح
خانه‌های	خانه‌ای
مدرسه‌های	مدرسه‌ای
فایده‌های	فایده‌ای
ذره‌های	ذره‌ای
جامعه‌های	جامعه‌ای
عده‌های	عده‌ای
اندازه‌های	اندازه‌ای
بچه‌های	بچه‌ای
غریبه‌های	غریبه‌ای
قهوه‌های	قهوه‌ای
ویژه‌های	ویژه‌ای
خاطره‌های	خاطره‌ای

جدول ۴-۴: نتایج پالایش داده‌ها

بخش	نمونه اولیه	حذف شده	نمونه نهایی
آموزش (train)	۲۹۷۸۹	۶۵۷	۲۹۱۳۲
اعتبارسنجی (validation)	۱۰۶۷۶	۳۴۹	۱۰۳۲۷
آزمون (test)	۱۰۶۷۶	۵۳۶	۱۰۱۴۰

۴-۳-۳ مجموعه داده نهایی

پس از پالایش، مجموعه داده نهایی شامل سه ستون اصلی بود: متن مرجع نرمال شده، خروجی Whisper نرمال شده و خروجی MMS نرمال شده.

۴-۴ پیاده سازی و آموزش

۴-۴-۱ پیکربندی مدل

مدل پیشنهادی با تنظیمات جدول ۴-۵ پیاده سازی شد.

۴-۴-۲ لایه تعبیه کاراکتری یا زیرواژه ای

لایه تعبیه (Embedding) وظیفه تبدیل شاخص های گسسته داخل دیکشنری به بردارهای پیوسته را بر عهده دارد. این لایه با ابعاد واژگان در بُعد تعبیه تعریف می شود. در این پروژه، بُعد تعبیه برابر با ۱۲۸ در نظر گرفته شده است.

این لایه یک پارامتر قابل آموزش است. بردارهای تعبیه به صورت تصادفی مقداردهی اولیه می شوند و سپس با گرادینان کاهشی بهینه می شوند.

۴-۴-۳ دیکشنری کاراکتری

برای مدل های کاراکتری، دیکشنری از تمام کاراکترهای موجود در سه ستون متن مرجع، خروجی Whisper و خروجی MMS ساخته شد. فرآیند ساخت دیکشنری شامل پنج مرحله است: جمع آوری کاراکترها، شمارش فراوانی، فیلتر کاراکترهایی با فراوانی کمتر از ۲، مرتب سازی الفبایی، و افزودن توکن های خاص (<pad>، <sos>، <eos> و نماد هم تراز) به ابتدای لیست. دیکشنری نهایی شامل ۵۲ کاراکتر بود.

۴-۴-۴ تفاوت مدل کاراکتری و زیرواژه ای

مدل کاراکتری: هر کاراکتر یک واحد مستقل است. دیکشنری شامل ۵۲ کاراکتر و آموزش هر epoch حدود ۳۰۰ ثانیه (پنج دقیقه) زمان می برد.

مدل زیرواژه ای: از قطعه بندی BPE یا SentencePiece استفاده می کند. دیکشنری شامل ۱۲۸۰ زیرواژه است.

تحلیل تفاوت زمان آموزش: در مدل کاراکتری، طول دنباله ورودی برابر با تعداد کاراکترهای متن است؛ مثلاً یک جمله ۵۰ کاراکتری دارای دنباله ورودی با طول ۵۰ است. در مدل زیرواژه ای، همین جمله به ۲۰ تا ۳۰ زیرواژه تبدیل می شود. از آنجا که پیچیدگی محاسباتی LSTM به صورت خطی با طول دنباله افزایش می یابد،

کاهش طول دنباله مستقیماً به کاهش زمان آموزش منجر می‌شود. به همین دلیل، هر epoch در مدل زیرواژه‌ای حدود ۶۰ تا ۹۰ ثانیه طول می‌کشد.

با این حال، مدل زیرواژه‌ای یک ایراد دارد: اگر یک زیرواژه به‌اشتباه تولید شود، ممکن است چندین کاراکتر را به‌طور همزمان تحت تأثیر قرار دهد و CER را به‌طور نامتناسب افزایش دهد. به همین دلیل، مدل کاراکتری به‌عنوان مدل اصلی انتخاب شد.

۴-۴-۵ روش‌های کاهش نرخ Teacher Forcing

در آموزش مدل‌های دنباله‌به‌دنباله، تکنیک Teacher Forcing برای تسریع همگرایی استفاده می‌شود. در این تکنیک، در هر گام رمزگشایی، به‌جای استفاده از خروجی مدل در گام قبل، از توکن صحیح متن مرجع استفاده می‌شود. با این حال، استفاده مداوم از Teacher Forcing می‌تواند به انحراف آموزش-استنتاج (Exposure Bias) منجر شود؛ به این معنا که مدل در زمان آموزش به دیدن توکن‌های صحیح عادت می‌کند و در زمان استنتاج که باید از خروجی خودش استفاده کند، دچار افت عملکرد می‌شود.

اگر نرخ Teacher Forcing خیلی کم باشد، مدل در مراحل اولیه آموزش نمی‌تواند الگوهای صحیح را بیاموزد و همگرایی به‌کندی انجام می‌شود. اگر نرخ Teacher Forcing با سرعت زیاد کاهش یابد، مدل ممکن است پیش از یادگیری الگوهای پایه، با خروجی‌های نادرست خود مواجه شود و آموزش ناپایدار گردد. اگر نرخ Teacher Forcing با سرعت خیلی کم کاهش یابد، مدل بیش‌ازحد به توکن‌های صحیح وابسته می‌شود و در زمان استنتاج دچار مشکل می‌شود.

در این پروژه، از دو روش مختلف برای کاهش نرخ Teacher Forcing استفاده شد:

روش نخست — کاهش خطی. در این روش، نرخ Teacher Forcing به‌صورت خطی از مقدار حداکثر به مقدار حداقل کاهش می‌یابد:

$$TF(e) = \max(TF_{\max} - (e - 1) \cdot TF_{\text{decay}}, TF_{\min}) \quad (4-1)$$

که در آن $TF_{\max} = 7.0$ ، $TF_{\min} = 0.0$ و $TF_{\text{decay}} = 0.05$ است. این روش در مدل کاراکتری استفاده شد. مزیت این روش، سادگی پیاده‌سازی و کنترل دقیق نرخ کاهش است. ایراد آن، این است که کاهش خطی ممکن است برای مراحل اولیه خیلی سریع و برای مراحل پایانی خیلی کند باشد.

روش دوم — کاهش سیگموئیدی. در این روش، نرخ Teacher Forcing با استفاده از تابع سیگموئید کاهش می‌یابد:

$$TF(e) = TF_{\min} + (TF_{\max} - TF_{\min}) \cdot \frac{1}{1 + e^{\lambda \cdot (e - e_0)}} \quad (۲-۴)$$

این روش در مدل با دیشکری زیرواژه ای استفاده شده است. مزیت این روش، کاهش نرم و تدریجی در ابتدا و انتها، و کاهش سریع‌تر در میانه آموزش است که با مرحله حساس یادگیری مدل هماهنگ است. این روش باعث می‌شود در ابتدای آموزش که مدل به راهنمایی بیشتری نیاز دارد، نرخ Teacher Forcing بالا بماند و در میانه آموزش که مدل شروع به یادگیری الگوهای مستقل می‌کند، نرخ با شیب تندتری کاهش یابد.

۴-۴-۶ Dropout حسی

یکی از چالش‌های اصلی در وظیفه تصحیح پس از ASR، وابستگی بیش‌ازحد مدل به یک نما است. اگر مدل یاد بگیرد که همیشه به خروجی یک ASR خاص اتکا کند، در مواقعی که آن ASR دچار خطا شود، مدل نیز دچار خطا می‌شود و مکانیزم ترکیب چندنمایی بی‌اثر می‌گردد. برای جلوگیری از این مشکل، در این پروژه از تکنیک Dropout حسی استفاده شد. در این تکنیک، در هر مرحله آموزش، با احتمال مشخص، یکی از دو نمای ورودی به‌طور تصادفی حذف یا بخشی از آن ماسک می‌شود. این کار باعث می‌شود که مدل یاد بگیرد به‌تنهایی از هر نما نیز اطلاعات استخراج کند و به یک نما وابسته نشود.

دو حالت برای Dropout حسی پیاده‌سازی شد:

- حالت کامل: در این حالت، کل یکی از دو نما (به‌طور تصادفی) با توکن‌های <pad> جایگزین می‌شود.
- حالت جزئی: در این حالت، بخشی از یکی از دو نما (به‌طور تصادفی و با طول ۲ تا ۵ کاراکتر) با توکن‌های <pad> جایگزین می‌شود.

در این پروژه، حالت جزئی به‌عنوان حالت پیش‌فرض انتخاب شد. در حالت جزئی، یک بخش تصادفی از یکی از نماها با توکن پرکننده جایگزین می‌شود که این کار باعث می‌شود مدل یاد بگیرد از اطلاعات جزئی موجود در هر نما استفاده کند.

۴-۴-۷ Dropout در سایر بخش‌ها

علاوه بر Dropout حسی، از Dropout معمولی در بخش‌های زیر استفاده شد:

- رمزگذار BiLSTM

- رمزگشای LSTM

- لایه خطی خروجی

- بلوک ادغام گیت دار

۸-۴-۴ مقادیر هایپرپارامترها

انتخاب مقادیر هایپرپارامترها بر اساس ملاحظات زیر انجام شد:

- نرخ یادگیری $1e-4$: نرخ یادگیری نسبتاً پایین انتخاب شد تا مدل به سرعت دچار بیش برازش (Overfitting) نشود.

- وزن کاهش $1e-5$: وزن کاهش کوچک برای جلوگیری از بیش برازش استفاده شد.

- اندازه دسته ۶۴: تعادل مناسبی بین سرعت آموزش و پایداری گرادیان فراهم می کند.

- تعداد لایه ها ۲: تعداد لایه های ۲ برای رمزگذار و رمزگشا انتخاب شد تا ظرفیت مدل با حجم داده ها متناسب باشد.

- نرخ Dropout ۰.۵: نرخ نسبتاً بالا برای کاهش بیش برازش روی مجموعه داده های کوچک.

- کلیپ گرادیان ۱.۰: برای جلوگیری از انفجار گرادیان و ناپایداری آموزش.

۹-۴-۴ سخت افزار و زمان آموزش

آموزش مدل روی GPUهای T۴ در پلتفرم Kaggle انجام شد. هر epoch آموزش در مدل کاراکتری حدود ۳۰۰ ثانیه (پنج دقیقه) و در مدل زیرواژه ای حدود ۶۰ تا ۹۰ ثانیه زمان برد. در مجموع، با احتساب توقف زودهنگام، آموزش مدل در حدود ۶۰ تا ۸۰ epoch متوقف می شد.

۱۰-۴-۴ ذخیره سازی و بازیابی

در طول آموزش، مکانیزم نقطه بازرسی (checkpoint) در فواصل هر ۱۰ epoch پیاده سازی شد. در هر نقطه بازرسی، وزن های مدل، وضعیت بهینه ساز و بهترین CER ذخیره می شد. این کار به دو دلیل انجام شد: نخست، در صورت قطع شدن آموزش، می توان آن را از آخرین نقطه بازرسی ادامه داد؛ دوم، امکان مقایسه مدل های ذخیره شده در epoch های مختلف فراهم می شود.

۴-۵ تحلیل خطا و ماهیت نویز

۴-۵-۱ ابزارهای تحلیل خطا

برای تحلیل ماهیت نویز در خروجی‌های ASR و عملکرد مدل، یک ابزار تحلیل خطای جامع پیاده‌سازی شد که شامل پنج بخش اصلی است: تحلیل آماری کلی (شامل محاسبه CER و WER میانگین و میانه)، تحلیل همبستگی خطا (محاسبه ضریب همبستگی پیرسون بین الگوهای خطای Whisper و MMS)، تحلیل تاکسونومی خطا (دسته‌بندی نمونه‌ها به پنج گروه)، تحلیل خطاهای کاراکتری (دسته‌بندی به جانشینی، درج و حذف) و تحلیل خطاهای سطح کلمه (دسته‌بندی به Near Miss، Semantic Swap، درج، حذف و هم‌آوا).

۴-۵-۲ نتایج تحلیل خطا

نتایج تحلیل خطا بر روی مجموعه‌آزمون (۱۰۱۴۱ نمونه) در جدول ۴-۶ خلاصه شده است.

۴-۵-۳ تحلیل هم‌آوایی در زبان فارسی

یکی از دلایل کلیدی که خطاهای ASR روی زبان فارسی کاملاً سیستماتیک نیستند، هم‌آوایی گسترده در خط فارسی است. در زبان فارسی، چندین نویسه مختلف دارای تلفظ یکسان هستند:

• صدای /z/: «ز»، «ذ»، «ض» و «ظ» — «حضرت» را می‌توان به اشتباه «حذرت» یا «حزرت» یا «حظرت» نوشت.

• صدای /s/: «س»، «ص» و «ث» — «صادرات» را می‌توان «سادات» نوشت.

• صدای /t/: «ت» و «ط» — «انتقاد» را می‌توان «انطقاد» نوشت.

• صدای /h/: «ه» و «ح» — «حسین» را می‌توان «هسین» نوشت.

• صدای /gh/: «ق» و «غ» — «قضایی» را می‌توان «غضایی» نوشت.

در زبان عربی، این نویسه‌ها تلفظ‌های متفاوتی دارند (مثلاً «ذ» و «ز» در عربی متفاوت تلفظ می‌شوند)، بنابراین یک مدل ASR می‌تواند تمایز بین آن‌ها را از ویژگی‌های صوتی بیاموزد. اما در زبان فارسی، تمام این نویسه‌ها یکسان تلفظ می‌شوند و تمایز بین آن‌ها صرفاً از روی شکل نوشتاری قابل تشخیص است. این تمایز یک امر حفظی است که فارسی‌زبانان به‌مرور با دیدن شکل نوشتاری کلمات یاد می‌گیرند. برای نمونه، مدل‌ها اغلب کلمه «حضرت» را با یک حرف «ز» دیگر که تکرار آماری بیشتری در کلمات مشابه داشته (به صورت حذرت)، می‌نویسند؛ اما ممکن است به‌طور تصادفی به صورت «حزرت» یا «حظرت» نیز نوشته شود. این مسئله ماهیت خطاها را از حالت سیستماتیک به حالت تصادفی نزدیک‌تر می‌کند.

جدول ۴-۵: پیکربندی مدل پیشنهادی

پارامتر	مقدار
بُعد تعبیه کاراکتری	۱۲۸
بُعد پنهان رمزگذار (BiLSTM دوسطحی)	۱۲۸ در هر جهت
بُعد پنهان رمزگشا (LSTM دوسطحی)	۲۵۶
تعداد لایه‌های رمزگذار	۲
تعداد لایه‌های رمزگشا	۲
نرخ Dropout	۰.۵
بهینه‌ساز	Adam
نرخ یادگیری اولیه	$1e-4$
وزن کاهش	$1e-5$
اندازه دسته	۶۴
حداکثر epoch	۱۵۰
کلیپ گرادین	۱.۰
معیار انتخاب مدل	کمترین CER
صبر (Patience)	epoch ۲۰

جدول ۴-۶: نتایج تحلیل خطا بر روی مجموعه آزمون

مقدار بر حسب درصد	معیار
۲۵.۸۷	دقت Whisper (تطابق دقیق)
۲۸.۹۱	دقت MMS
۲۹.۶۵	دقت مدل ترکیبی
۳۹.۸۹	Oracle نظری
۳۷.۳۳	همبستگی خطای Whisper و MMS
۱۴.۸۹	هر دو مدل درست (Both Correct)
۱۴.۰۲	فقط MMS درست (Only MMS Correct)
۱۰.۹۸	فقط Whisper درست (Only Whisper Correct)
۱.۷۸	هر دو اشتباه، مدل درست (Both Failed Fusion Succeeded)
۵۸.۳۳	هر دو اشتباه، مدل اشتباه (Both Failed Fusion Failed)

جدول ۴-۷: توزیع خطاهای کاراکتری و سطح کلمه

درصد	تعداد	نوع خطا
خطاهای کاراکتری		
۴۵.۱۷	۱۱۷۵۴	جانشینی
۱۴.۸۳	۱۷۴۳	هم‌آوا
۶۲.۶۲	۷۳۶۰	غیرهم‌آوا
۲۸.۷۲	۷۴۷۴	حذف
۲۶.۱۱	۶۷۹۴	درج
خطاهای سطح کلمه		
۴۶.۷۸	۸۷۲۳	نزدیک به خطا (Near Miss)
۲۵.۹۶	۴۸۴۰	جانشینی معنایی (Semantic Swap)
۱۱.۹۳	۲۲۲۴	درج
۹.۶۵	۱۸۰۰	حذف
۵.۶۸	۱۰۶۰	هم‌آوا

فصل پنجم

نتایج تجربی

۵-۱ مقدمه فصل

در این فصل، نتایج تجربی حاصل از ارزیابی مدل‌های مختلف پیشنهادی روی مجموعه داده استاندارد گفتار فارسی Common Voice ارائه و تحلیل می‌شود. برخلاف برخی از گزارش‌های مشابه که صرفاً به مقایسه عددی نتایج بسنده می‌کنند، این فصل تلاش می‌کند علت شکست رویکرد ترکیب چندنمایی برای زبان فارسی را شرح دهد. ساختار این فصل به گونه‌ای طراحی شده که خواننده ابتدا با نتایج عددی مدل‌های مختلف آشنا شود، سپس با تحلیل رفتار گیت‌ها مواجه گردد، و در نهایت با استدلال‌های نظری و آماری قانع شود که چرا معماری مطرح‌شده در فصل سوم، به‌رغم طراحی دقیق و پیاده‌سازی صحیح، برای زبان فارسی نتایج کاملاً مطلوبی به همراه نداشته است.

۵-۲ نتایج کمی مدل‌ها

۵-۲-۱ خطوط پایه

ابتدا عملکرد دو مدل ASR پایه روی مجموعه تست مورد ارزیابی قرار گرفت. این دو مدل به‌عنوان نمای اول و نمای دوم در تمام آزمایش‌های این فصل استفاده شدند. جدول ۵-۱ این نتایج را نشان می‌دهد.

در زبان فارسی MMS به‌طور معناداری از **Whisper** بهتر عمل می‌کند. MMS بر پایه wav2vec 2.0 بنا شده که

جدول ۵-۱: عملکرد خطوط پایه روی مجموعه تست

مدل	CER	WER	دقت تطابق دقیق	تعداد نمونه‌های درست
Whisper-large-v3	۰.۰۹۸۰	۰.۳۲۸۳	۲۵/۸۷٪	۲۶۲۳
MMS-1B-All	۰.۰۷۰۲	۰.۲۸۴۱	۲۸/۹۱٪	۲۹۳۲

از یادگیری خودنظارتی (Self-Supervised) روی سیگنال خام صوتی بهره می‌برد. این نوع یادگیری، برخلاف معماری رمزگذار-رمزگشای Whisper که بر پیش‌بینی توکن‌های متن تمرکز دارد، در شرایط داده کم منبع مزیت محسوسی دارد زیرا به حجم عظیمی از متن برچسب‌خورده نیاز ندارد.

این یافته، نقطه شروع مهمی برای تحلیل‌های بعدی است: از آنجا که MMS از ابتدا CER پایینی دارد، فضای بهبود برای یک مدل ترکیبی در زبان فارسی محدود است — برخلاف زبان راجستانی که در آن، هر دو مدل CER اولیه بالایی دارند و بنابراین ترکیب، پتانسیل بالایی برای بهبود دارد.

۵-۲-۲ مدل زیرواژه‌ای

مدل زیرواژه‌ای بر پایه قطعه‌بندی SentencePiece با ۱۲۸۰ زیرواژه آموزش داده شد. در این مدل، از راهبرد هم‌ترازی کاراکتر تهی استفاده شد. نتایج روی مجموعه تست در جدول ۵-۲ آمده است.

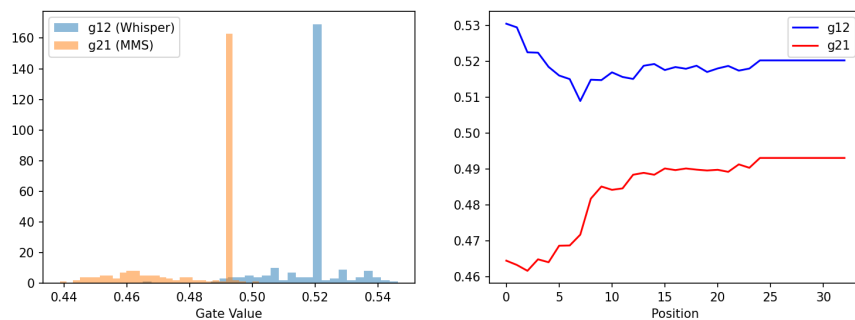
جدول ۵-۲: عملکرد مدل زیرواژه‌ای روی مجموعه تست

مقدار	معیار
۲/۶۶۳۷	Loss نهایی
۰/۱۸۹۷	CER
۰/۳۹۹۸	WER
۹۳/۶۲٪- (بدتر)	بهبود نسبت به Whisper
۱۷۰/۲۰٪- (بدتر)	بهبود نسبت به MMS

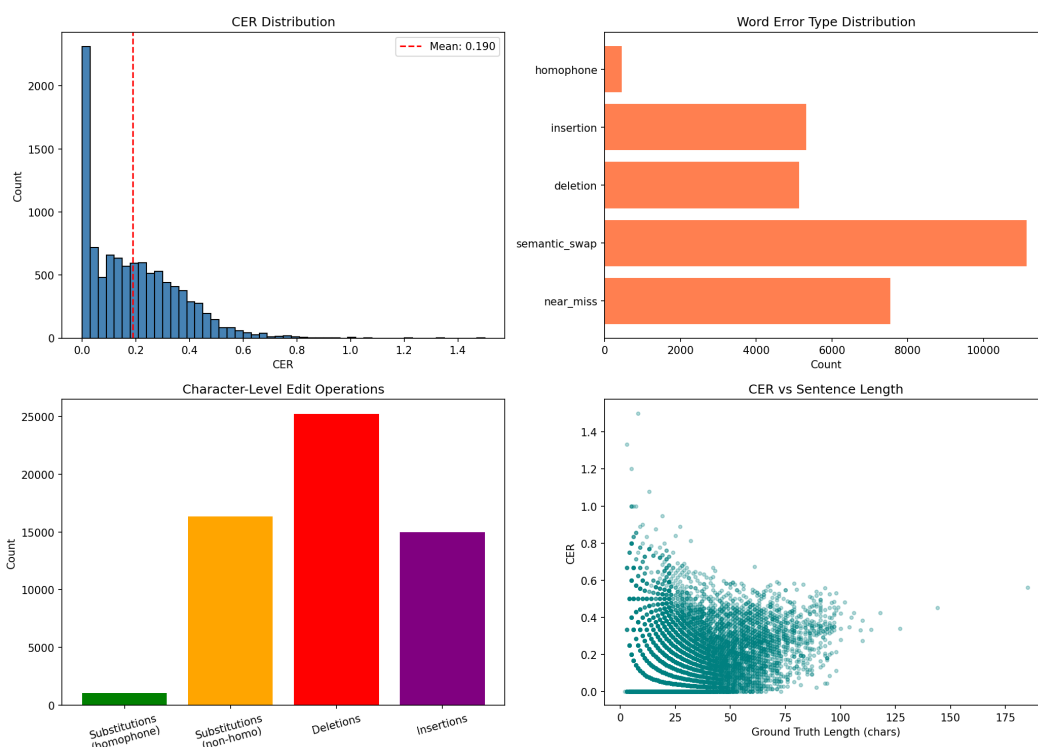
مدل زیرواژه‌ای شکست کامل خورد — CER آن تقریباً دو برابر Whisper CER خام است و از هر دو خط پایه به‌طور معناداری بدتر عمل می‌کند. دلایل این شکست را می‌توان به دو عامل اصلی نسبت داد:

نخست، تولید نادرست یک زیرواژه، چندین کاراکتر را به‌طور همزمان تحت تأثیر قرار می‌دهد. در سطح کاراکتر، اگر مدل یک کاراکتر را نادرست تولید کند، تنها CER به اندازه $1/|y|$ افزایش می‌یابد که $|y|$ طول متن مرجع است. اما در سطح زیرواژه، اگر مدل یک زیرواژه نادرست تولید کند که معادل سه کاراکتر است، CER به اندازه $3/|y|$ افزایش می‌یابد — یعنی سه برابر بیشتر. این اثر تجمعی در جملات طولانی، به‌ویژه در حضور زیرواژه‌های چندکاراکتری، مقدار CER را به‌طور نامتناسب افزایش می‌دهد.

دوم، ۱۲۸۰ زیرواژه برای پوشش کامل نگارش فارسی کافی نیست. یک دیکشنری زیرواژه‌ای با این حجم، نمی‌تواند همه ترکیب‌های نگارشی رایج را در بر بگیرد.



شکل ۵-۱: نمودار گیت‌های مدل زیرواژه‌ای. نمودار سمت چپ توزیع مقادیر گیت را نشان می‌دهد و نمودار سمت راست تغییرات میانگین گیت با موقعیت کاراکتر در جمله را نمایش می‌دهد.



شکل ۵-۲: توزیع خطاهای مدل زیرواژه‌ای در چهار پل: توزیع CER، توزیع خطاهای سطح کلمه، عملیات ویرایش در سطح کاراکتر، و رابطه CER با طول جمله.

۵-۲-۳ مدل کاراکتری با هم‌ترازی کاراکتر تهی

بهترین مدل ترکیبی این پروژه، مدل کاراکتری با هم‌ترازی کاراکتر تهی است. در این مدل، از راهبرد نخست هم‌ترازی (درج نماد تهی $\emptyset$ در موقعیت‌هایی که یک کاراکتر در یکی از دو خروجی وجود ندارد) استفاده شد. این راهبرد، همان‌طور که پیش‌تر توضیح داده شد، اطلاعات تمایزی میان دو نما را حفظ می‌کند و اجازه می‌دهد مدل در هر موقعیت کاراکتری، تصمیم مستقلی درباره وزن‌دهی به هر نما بگیرد. نتایج این مدل در جدول ۵-۳ گزارش شده است.

این مدل دو یافته متناقض را نشان می‌دهد که تحلیل دقیق آن‌ها برای درک ماهیت مسئله ضروری است:

Whisper Baseline - CER: 0.0980, WER: 0.3283
 MMS Baseline - CER: 0.0702, WER: 0.2841
 Our Model - CER: 0.1897, WER: 0.3998

Improvement over Whisper:

CER: -93.62% reduction

WER: -21.77% reduction

شکل ۵-۳: نتایج CER و WER مدل زیرواژه‌ای روی مجموعه تست در مقایسه با خطوط پایه.

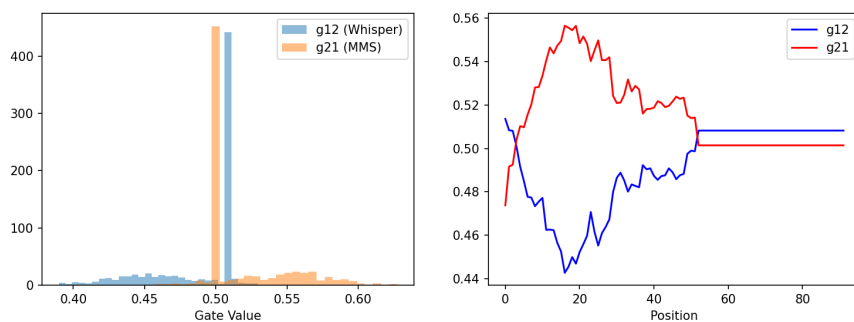
جدول ۵-۳: عملکرد مدل کاراکتری با هم‌ترازی کاراکتر تهی

معیار	مقدار	بهبود از Whisper	بهبود از MMS
Loss نهایی	۱/۸۹۷۰	—	—
CER	۰.۰۷۲۶	کاهش ۲۵/۹۳٪	افزایش ۳/۴۲٪
WER	۰.۲۸۵۵	کاهش ۱۳/۰۴٪	افزایش ۰/۴۹٪
دقت تطابق دقیق	۲۹/۶۵٪	۱۴/۶۰٪+	۲/۵۶٪+

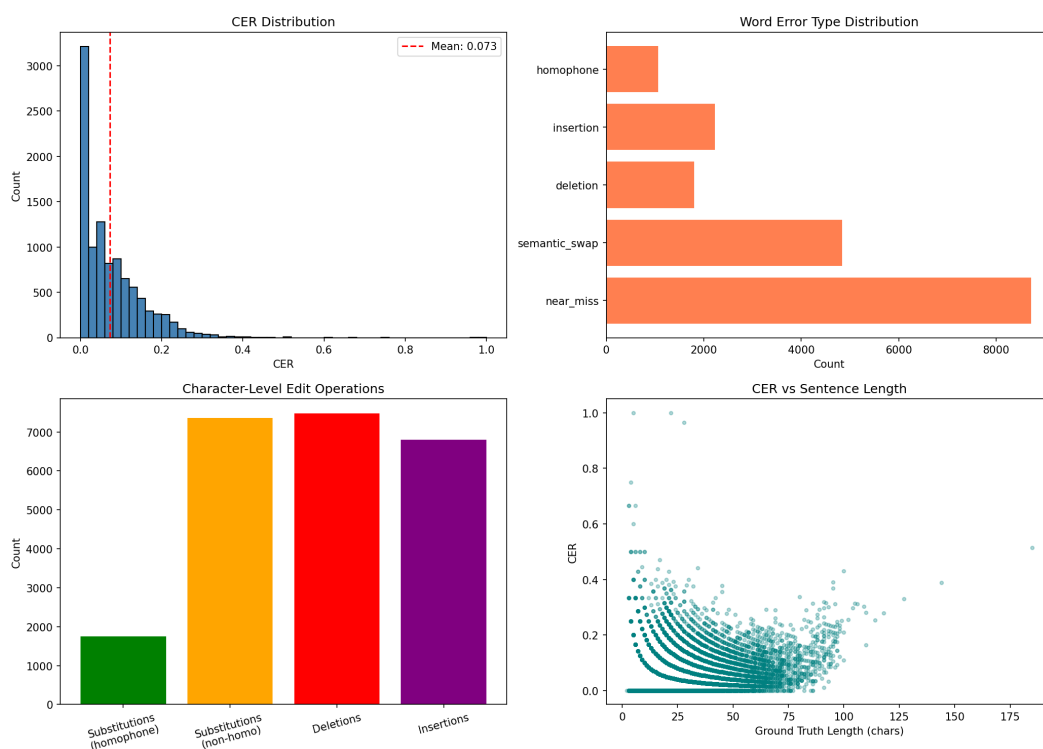
یافته اول — بهبود قابل توجه نسبت به **Whisper**: مدل ترکیبی توانست CER را از ۰.۰۹۸۰ به ۰.۰۷۲۶ کاهش دهد. این بهبود در هر دو معیار CER و WER مشاهده می‌شود و نشان می‌دهد که بخش رمزگذار-رمزگشای مدل، توانایی تصحیح خروجی Whisper را دارد. این بخش، مستقل از مکانیزم گیت عمل می‌کند و به‌عنوان یک مدل زبانی کوچک، الگوهای نگارشی رایج فارسی را می‌آموزد.

یافته دوم — عدم بهبود نسبت به **MMS**: با این حال، همین مدل ترکیبی از MMS خام (۰.۰۷۰۲) اندکی بدتر است. این یافته نشان می‌دهد مکانیزم گیت در این مدل نقش مثبتی ایفا نکرده و بخش بزرگی از بهبود مشاهده‌شده، ناشی از بخش رمزگذار-رمزگشا است، نه گیت.

یافته سوم — بهبود در تطابق دقیق: نکته جالب آن است که در معیار تطابق دقیق (Exact Match)، مدل ترکیبی با ۲۹/۶۵٪ از هر دو خط پایه بهتر است (Whisper: ۲۵/۸۷٪ و MMS: ۲۸/۹۱٪). این پارادوکس ظاهری در بخش‌های آینده به‌طور کامل تحلیل خواهد شد.



شکل ۴-۵: نمودار گیت‌های مدل کاراکتری با کاراکتر تهی. در این مدل، توزیع گیت‌ها پهن‌تر از مدل زیرواژه‌ای است که نشان‌دهنده پویایی بیشتر مکانیزم گیت است.



شکل ۵-۵: توزیع خطاهای مدل کاراکتری با کاراکتر تهی. مقایسه این نمودار با شکل ۵-۲ نشان می‌دهد که مدل کاراکتری خطاهای کمتری در همه دسته‌ها تولید کرده است.

۴-۲-۵ مدل کاراکتری با هم‌ترازی رشته طولانی‌تر

استفاده از راهبرد هم‌ترازی رشته طولانی‌تر نتایج ضعیف‌تری به همراه داشت. در این راهبرد، به‌جای درج نماد تهی، کاراکتر متفاوت در دنباله کوتاه‌تر با کاراکتر متناظر در دنباله طولانی‌تر جایگزین می‌شود. نتایج در جدول ۴-۵ آمده است.

این یافته نشان می‌دهد راهبرد جایگزینی کاراکتر از رشته طولانی‌تر، اطلاعات تمایزی میان دو نما را از بین می‌برد و به تحمیل خطاهای یک نما به نمای دیگر منجر می‌شود.

Whisper Baseline - CER: 0.0980, WER: 0.3283
 MMS Baseline - CER: 0.0702, WER: 0.2841
 Our Model - CER: 0.0726, WER: 0.2855

Improvement over Whisper:

CER: 25.93% reduction

WER: 13.04% reduction

شکل ۵-۶: نتایج CER و WER مدل کاراکتری با کاراکتر تهی روی مجموعه تست.

جدول ۵-۴: عملکرد مدل کاراکتری با هم‌ترازی رشته طولانی‌تر

معیار	مقدار	Δ از راهبرد کاراکتر تهی
Loss نهایی	۱/۸۳۰۹	-۶۶۱٪
CER	۰.۰۸۳۰	+۱۰۴٪
WER	۰.۳۱۳۰	+۲۷۵٪

۵-۲-۵ مدل کاراکتری با فرمول گیت‌های مقاله و Gate Variance

برای بررسی اثر افزودن Gate Variance Regularization و بازگشت به فرمول اصلی مقاله، دو آزمایش اضافی انجام شد. در آزمایش اول، از فرمول اصلی مقاله (Residual Modulator با tanh) به‌عنوان مکانیزم گیت استفاده شد. در آزمایش دوم، از فرمول ترکیب محدب (Convex Combination) که در پیاده‌سازی اولیه این پروژه استفاده شده بود، بهره برده شد. در هر دو آزمایش، Gate Variance Regularization با ضریب $\lambda = 0.1$ به تابع هزینه اضافه شد. نتایج در جدول ۵-۵ آمده است.

جدول ۵-۵: نتایج مدل‌های با Gate Variance Regularization

مدل	فرمول گیت	CER	WER	توقف
Paper + Gate Var	Residual Modulator	۰.۰۷۷۴	۰.۳۰۲۶	۷۱ epoch
Convex + Gate Var	Convex Combination	۰.۰۷۸۶	۰.۳۰۶۲	۵۹ epoch

نتیجه کلیدی: افزودن Gate Variance، برخلاف انتظار اولیه، باعث بدتر شدن نتایج شد. مقایسه با بهترین مدل (کاراکتری با کاراکتر تهی، CER = ۰.۰۷۲۶) نشان می‌دهد که این دو آزمایش، به‌ترتیب ۶/۶٪ و ۸/۳٪ بدتر عمل کردند. این یافته به‌ظاهر متناقض، در بخش ۵-۳ به‌طور عمیق تحلیل خواهد شد، زیرا نشان می‌دهد که

فروپاشی گیت یک علامت (Symptom) از مشکل عمیق‌تر است، نه علت آن.

Whisper Baseline - CER: 0.0980, WER: 0.3283

MMS Baseline - CER: 0.0702, WER: 0.2841

Our Model - CER: 0.0774, WER: 0.3026

شکل ۵-۷: نتایج CER و WER مدل کاراکتری با فرمول مقاله و Gate Variance.

Whisper Baseline - CER: 0.0980, WER: 0.3283

MMS Baseline - CER: 0.0702, WER: 0.2841

Our Model - CER: 0.0786, WER: 0.3062

شکل ۵-۸: نتایج CER و WER مدل کاراکتری با فرمول ترکیب محدب و Gate Variance.

۵-۲-۶ جدول جامع مقایسه

جدول ۵-۶ نتایج همه هفت مدل را در یک نگاه جمع‌بندی می‌کند.

جدول ۵-۶: مقایسه جامع همه مدل‌ها روی مجموعه تست

مدل	CER	WER	Whisper CER Δ	MMS CER Δ
Whisper خام	۰.۰۹۸۰	۰.۳۲۸۳	—	٪۲۷۸+
MMS خام	۰.۰۷۰۲	۰.۲۸۴۱	٪۲۷۸-	—
زیرواژه‌ای	۰.۱۸۹۷	۰.۳۹۹۸	٪۹۱۷+	۰/۱۱۹۵+
کاراکتری + کاراکتر تهی	۰.۰۷۲۶	۰.۲۸۵۵	٪۲۵۴-	٪۰۲۴+
کاراکتری + رشته طولانی‌تر	۰.۰۸۳۰	۰.۳۱۳۰	٪۱۵۰-	٪۱۲۸+
Paper + Gate Var	۰.۰۷۷۴	۰.۳۰۲۶	٪۲۰۶-	٪۰۷۲+
Convex + Gate Var	۰.۰۷۸۶	۰.۳۰۶۲	٪۱۹۴-	٪۰۸۴+

۵-۳ تحلیل فروپاشی گیت

۵-۳-۱ تعریف و مکانیزم فروپاشی گیت

فروپاشی گیت (Gate Collapse) پدیده‌ای است که در آن گیت‌های یک مدل ترکیب، به‌جای یادگیری تصمیمات

پویا و وابسته به محتوا، به یک مقدار تقریباً ثابت همگرا می‌شوند. در معنای دقیق ریاضی، اگر $g_{12,t}$ مقدار گیت

در گام زمانی t باشد، فروپاشی رخ می‌دهد اگر:

$$\text{Var}_{t \sim \mathcal{T}}(g_{12,t}) \rightarrow 0 \quad (۱-۵)$$

یعنی واریانس گیت در طول زمان به صفر میل کند. در این حالت، گیت هیچ اطلاعاتی درباره محتوا منتقل نمی‌کند و مکانیزم ترکیب، عملاً به یک میانگین‌گیری ساده با وزن ثابت تبدیل می‌شود.

فروپاشی گیت به دلیل ترکیب چند عامل رخ می‌دهد:

الف) اشباع تابع سیگموئید: تابع سیگموئید $\sigma(z) = \frac{1}{1+e^{-z}}$ در نواحی $z \gg 0$ و $z \ll 0$ به شدت اشباع می‌شود. در این نواحی، مشتق تابع $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ بسیار کوچک است و گرادینانها به سختی منتقل می‌شوند. اگر گیت در این نواحی قرار بگیرد، یادگیری متوقف می‌شود.

ب) محو شدن گرادینانها در دنباله‌های طولانی: در مدل‌های بازگشتی، گرادینانها با عبور از گام‌های زمانی متعدد، به تدریج کوچک می‌شوند. در نتیجه، گیت‌های مربوط به گام‌های اولیه، به سختی آموزش می‌بینند و به مقدار اولیه خود نزدیک می‌مانند.

ج) تمایل به ساده‌ترین راه‌حل: در مسیر کمینه‌سازی loss، مدل ابتدا ساده‌ترین راه‌حل — یعنی یک تابع وابسته به موقعیت با پارامترهای کم — را کشف می‌کند. پس از آن، گرادینانها در ناحیه راه‌حل محتوایی (که پیچیده‌تر است) بسیار کوچک می‌شوند و مدل هرگز به آن سمت نمی‌رود. این پدیده در ادبیات یادگیری ماشین با نام Shortcut Learning شناخته می‌شود.

معنای دقیق مقادیر گیت: برای درک معنای دقیق مقادیر گیت، ابتدا باید فرمول ترکیب را یادآوری کنیم. در معماری این پروژه، خروجی گیت‌دار به صورت زیر محاسبه می‌شود:

$$\hat{h}_t^{(1)} = (1 - g_{12,t}) \cdot v_t^{(1)} + g_{12,t} \cdot v_t^{(2)} \quad (۲-۵)$$

$$\hat{h}_t^{(2)} = g_{21,t} \cdot v_t^{(1)} + (1 - g_{21,t}) \cdot v_t^{(2)} \quad (۳-۵)$$

که در آن $v_t^{(1)}$ بازنمایی نمای Whisper و $v_t^{(2)}$ بازنمایی نمای MMS است. بر اساس این فرمول:

• $g_{12} = 0.1$: نمای Whisper تقریباً کاملاً به خودش اتکا می‌کند و تنها ۱۰٪ از اطلاعات نمای MMS را جذب می‌کند. یعنی مدل در این موقعیت، Whisper را مطمئن‌تر از MMS می‌داند.

• $g_{12} = 0.5$: نمای Whisper به‌طور مساوی از خودش و از MMS اطلاعات می‌گیرد. این وضعیت نشان‌دهنده عدم قطعیت مدل است و در نگاه اول می‌تواند نشان‌دهنده یک تصمیم متعادل باشد.

- $g_{12} = 0.9$: نمای Whisper تقریباً از MMS کپی می‌کند و تنها ۱۰٪ از اطلاعات خودش را نگه می‌دارد. یعنی مدل در این موقعیت، MMS را مطمئن‌تر از Whisper می‌داند.

به‌طور مشابه، برای g_{21} (که نقش معکوس دارد):

- $g_{21} = 0.1$: نمای MMS به خودش اتکا می‌کند.

- $g_{21} = 0.9$: نمای MMS از Whisper کپی می‌کند.

اگر گیت‌ها واقعاً محتوایی عمل کنند، انتظار داریم:

- در موقعیت‌هایی که Whisper درست و MMS غلط است، g_{12} به صفر نزدیک باشد (Whisper به خودش اتکا کند).

- در موقعیت‌هایی که Whisper غلط و MMS درست است، g_{12} به یک نزدیک باشد (Whisper از MMS کپی کند).

- در نمودار سمت چپ، یک توزیع پهن با مقادیر متنوع در کل بازه $[0, 1]$ مشاهده شود.

- در نمودار سمت راست، هیچ الگوی مشخصی با موقعیت وجود نداشته باشد.

اما اگر گیت‌ها فروپاشی کنند، انتظار داریم:

- در نمودار سمت چپ، یک پیک تیز در حدود 0.5 مشاهده شود.

- در نمودار سمت راست، یک الگوی مشخص (مثلاً کاهشی یا افزایشی) با موقعیت وجود داشته باشد.

- واریانس گیت در طول زمان بسیار کوچک باشد.

۵-۳-۲ شواهد تجربی فروپاشی در سه مدل

الف) مدل زیرواژه‌ای: نمودار گیت این مدل در شکل ۵-۱ نشان داده شده است. همان‌طور که مشاهده می‌شود، هر دو گیت g_{12} و g_{21} دارای توزیع تک‌پیک بسیار باریکی هستند که حول مقادیر ۰/۵۲ (برای g_{12}) و ۰/۴۹۵ (برای g_{21}) متمرکز شده‌اند. بیش از ۹۵٪ مقادیر در یک بازه ۴٪ واحدی قرار می‌گیرند که نشان‌دهنده فروپاشی شدید است. در نمودار سمت راست (تغییرات گیت با موقعیت)، الگوی کاهشی ضعیفی مشاهده می‌شود که از ۰/۵۳ به ۰/۵۲ در g_{12} و از ۰/۴۷ به ۰/۴۹ در g_{21} تغییر می‌کند — تغییری بسیار کوچک که تقریباً در حد نویز است.

(ب) مدل کاراکتری با کاراکتر تهی: این مدل توزیع پهن‌تری دارد (شکل ۵-۴). بازه تغییرات گیت از ۰/۴۰ تا ۰/۶۰ است که نشان می‌دهد مدل کاراکتری، به دلیل تفکیک‌پذیری بالاتر در سطح کاراکتر، کمی از فروپاشی دورتر شده است. در نمودار موقعیت، الگوی پیچیده‌تری مشاهده می‌شود: g_{12} ابتدا کاهش می‌یابد (از ۰/۵۱ به ۰/۴۴ در موقعیت ۲۰)، سپس به تدریج افزایش می‌یابد و در نهایت در ۰/۵۱ تثبیت می‌شود. g_{21} الگوی معکوسی دارد: ابتدا افزایش می‌یابد (از ۰/۴۷ به ۰/۵۶ در موقعیت ۲۰)، سپس کاهش می‌یابد و در ۰/۵۰ تثبیت می‌شود.

نکته کلیدی: این الگوی پیچیده‌تر، نشان می‌دهد که مدل کاراکتری کمی بهتر از زیرواژه‌ای عمل می‌کند، اما هنوز به فروپاشی کامل نزدیک است. دو گیت در موقعیت‌های ابتدایی متفاوت عمل می‌کنند، اما در موقعیت‌های انتهایی (بعد از ۶۰) به مقادیر ثابت ۰/۵۰ و ۰/۵۱ فروپاشی می‌کنند.

۵-۳-۳ Gate Variance Regularization: فرمول‌بندی کامل

برای مقابله با فروپاشی گیت، در این پروژه از تکنیک **Gate Variance Regularization** استفاده شد. ایده اصلی این تکنیک آن است که به جای انتظار برای یادگیری خودکار گیت‌های پویا، به‌طور صریح مدل را به تنوع در مقادیر گیت در طول زمان تشویق کنیم. برای این منظور، جمله‌ای به تابع هزینه اضافه می‌شود که واریانس گیت‌ها را به‌عنوان یک پاداش در نظر می‌گیرد.

فرمول‌بندی ریاضی: تابع هزینه کل به‌صورت زیر تعریف می‌شود:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} - \lambda \cdot (\text{Var}(g_{12}) + \text{Var}(g_{21})) \quad (۴-۵)$$

در این فرمول:

- $\mathcal{L}_{\text{CE}}$ تابع هزینه Cross-Entropy با Label Smoothing است که در بخش ۴-۴-۴ توضیح داده شد.
- λ ضریب وزنی است که میزان تأثیر جمله واریانس را کنترل می‌کند. در این پروژه، مقدار $\lambda = 0.1$ استفاده شد.

- $\text{Var}(g_{12})$ و $\text{Var}(g_{21})$ واریانس گیت‌های متقاطع در طول زمان هستند.

محاسبه واریانس گیت: واریانس هر گیت به‌صورت زیر محاسبه می‌شود:

$$\text{Var}(g_{12}) = \frac{1}{B \cdot H} \sum_{b=1}^B \sum_{h=1}^H \frac{1}{T} \sum_{t=1}^T \left(g_{12,t}^{(b,h)} - \bar{g}_{12}^{(b,h)} \right)^2 \quad (۵-۵)$$

که در آن:

• B اندازه دسته (Batch Size)

• H بُعد پنهان (Hidden Size) که در این پروژه ۱۲۸ است

• T طول دنباله (تعداد گام‌های زمانی)

• $g_{12,t}^{(b,h)}$ مقدار گیت g_{12} برای نمونه b ، در بُعد پنهان h ، در گام زمانی t

• $\bar{g}_{12}^{(b,h)}$ میانگین گیت g_{12} در طول زمان برای نمونه b و بُعد پنهان h

تفسیر ریاضی جمله واریانس: جمله $-\lambda \cdot \text{Var}(g_{12})$ در تابع هزینه، باعث می‌شود که کاهش واریانس گیت (فروپاشی) جریمه شود. به عبارت دیگر، مدل برای کاهش loss کل، باید هم Cross-Entropy را کم کند و هم واریانس گیت‌ها را بالا نگه دارد. این دو هدف در تنش با یکدیگرند و مدل باید تعادلی بین آن‌ها برقرار کند.

۴-۳-۵ توضیح Label Smoothing

Label Smoothing یک تکنیک منظم‌سازی است که در آموزش مدل‌های طبقه‌بندی استفاده می‌شود و هدف آن جلوگیری از بیش‌برازش (Overfitting) و بیش‌اطمینانی (Overconfidence) مدل است.

مسئله: در آموزش استاندارد با Cross-Entropy، توزیع هدف برای هر نمونه یک توزیع یک‌داغ (One-hot) است. یعنی اگر کاراکتر صحیح در موقعیت t ، حرف «ب» باشد، توزیع هدف به صورت زیر است:

$$y_{\text{target}} = [0, 0, \dots, 1, \dots, 0, 0] \quad (۶-۵)$$

که فقط در خانه مربوط به «ب» مقدار ۱ دارد و بقیه صفر هستند. این توزیع، مدل را تشویق می‌کند که احتمال پیش‌بینی «ب» را به سمت ۱ برود و احتمال بقیه کاراکترها را به سمت صفر. این رفتار در داده‌های آموزشی، منجر به بیش‌اطمینانی مدل می‌شود و در داده‌های آزمون، افت عملکرد ایجاد می‌کند.

راه‌حل **Label Smoothing**: در این تکنیک، به جای توزیع یک‌داغ، از یک توزیع نرم‌شده استفاده می‌شود.

اگر ϵ پارامتر Label Smoothing باشد (معمولاً ۰/۱)، توزیع هدف جدید به صورت زیر تعریف می‌شود:

$$y_{\text{smooth}} = (1 - \epsilon) \cdot y_{\text{one-hot}} + \frac{\epsilon}{V} \quad (۷-۵)$$

که در آن V تعداد کلاس‌ها (تعداد کاراکترهای واژگان) است. برای $\epsilon = 0.1$ و $V = 52$ ، توزیع هدف برای کاراکتر «ب» به صورت زیر خواهد بود:

$$y_{\text{smooth}} = [0.0019, 0.0019, \dots, 0.9, \dots, 0.0019, 0.0019] \quad (۸-۵)$$

یعنی به جای مقدار ۱ برای «ب» و صفر برای بقیه، حالا «ب» مقدار ۰/۹ و بقیه کاراکترها مقدار ۰/۱۹ می گیرند.

مزایای Label Smoothing:

- جلوگیری از بیش اطمینانی: مدل یاد می گیرد که پیش بینی های خود را با عدم قطعیت همراه کند.
- بهبود تعمیم پذیری: مدل در داده های آزمون عملکرد بهتری دارد.
- کاهش بیش برازش: مدل به جای حفظ کردن داده های آموزش، الگوهای عمومی تر را می آموزد.

در این پروژه، از Label Smoothing با $\epsilon = 0.1$ استفاده شد که منجر به بهبود پایداری آموزش و کاهش شکاف train/val گردید.

۵-۳-۵ سایر اصلاحات برای تقویت پویایی گیت

علاوه بر Gate Variance Regularization، سه اصلاح دیگر برای تقویت پویایی گیت ها اعمال شد: الف) Scaled Sigmoid: به جای استفاده از تابع سیگموئید خالص که خروجی آن در بازه (0, 1) قرار می گیرد، از یک نسخه مقیاس شده استفاده شد:

$$g_{12} = \alpha \cdot \sigma(W_{12}[v_1; v_2]) + \beta \quad (۹-۵)$$

که در آن $\alpha = 0.8$ و $\beta = 0.1$. این تغییر، گیت را در بازه باریک تر [0.1, 0.9] نگه می دارد و از اشباع شدید در نواحی ۰ و ۱ جلوگیری می کند.

ب) حذف Layer Normalization و بازگشت به فرمول مقاله: در فرمول اصلی مقاله از Layer Normalization استفاده نشده است. برای بررسی اثر این تغییر، در آزمایش جدید، Layer Normalization حذف شد و فرمول زیر جایگزین شد:

$$\hat{h}_t^{(1)} = v_t^{(1)} + g_{12,t} \odot \tanh(v_t^{(2)}) \quad (۱۰-۵)$$

$$\hat{h}_t^{(2)} = v_t^{(2)} + g_{21,t} \odot \tanh(v_t^{(1)}) \quad (۱۱-۵)$$

ج) Sensory Dropout کامل و جزئی: در نسخه اصلاح شده، Dropout حسی به دو حالت گسترش یافت: حالت کامل (با احتمال ۰/۱۵) که کل یک نما حذف می شود، و حالت جزئی (با احتمال ۰/۲۵) که بخشی از یک نما ماسک می شود.

با وجود اعمال همه این اصلاحات، CER مدل های جدید (۷۷۴٪ و ۷۸۶٪) از CER مدل پایه (۷۲۶٪) بدتر شد. این یافته به ظاهر متناقض، نشان دهنده یک پارادوکس عمیق است:

پارادوکس: Gate Variance Regularization واقعاً کار کرد (واریانس گیت‌ها از ۰.۱٪ به ۹۷٪ افزایش یافت)، اما نتایج را بدتر کرد.

این پارادوکس را می‌توان با سه استدلال توضیح داد:

استدلال اول — پویایی بی‌معنا: افزایش واریانس گیت، به‌خودی‌خود مفید نیست. آنچه اهمیت دارد، همبستگی گیت با درستی محتوایی است. اگر گیت‌ها پویا شوند اما پویایی‌شان تصادفی باشد، هیچ بهبودی حاصل نمی‌شود. استدلال دوم — تنش بین دو هدف: جمله $\lambda \cdot \text{Var}(g) - \text{مدل}$ را مجبور می‌کند که بین کاهش Cross-Entropy و افزایش واریانس تعادل برقرار کند. اگر λ بزرگ باشد، مدل ممکن است Cross-Entropy را فدا کند تا واریانس را بالا ببرد.

استدلال سوم — تقویت **Shortcut Learning**: مدل به‌جای یادگیری الگوهای محتوایی، به یک راه‌حل ساده‌تر پناه برد: نوسان گیت بر اساس موقعیت. این راه‌حل، هم واریانس بالا ایجاد می‌کند و هم Cross-Entropy را نسبی پایین نگه می‌دارد.

۴-۵ ماهیت موقعیتی گیت

۱-۴-۵ طرح آزمون و فرمول‌بندی

برای بررسی اینکه آیا گیت‌ها تصمیمات خود را بر اساس محتوا (کدام نما در آن موقعیت درست است) می‌گیرند یا بر اساس موقعیت کاراکتر در جمله، آزمون آماری زیر طراحی شد. روی ۵۷,۹۵۸ نقطه داده (شامل همه کاراکترهای همه نمونه‌های اعتبارسنجی)، چهار همبستگی محاسبه شد:

همبستگی اول: همبستگی بین مقدار گیت g_{12} و موقعیت کاراکتر در جمله:

$$r_{\text{pos}} = \text{Corr}(g_{12}, \text{position}) \quad (۱۲-۵)$$

همبستگی دوم: همبستگی بین مقدار گیت g_{12} و درستی نویسه در نمای MMS:

$$r_{\text{mms}} = \text{Corr}(g_{12}, \mathbb{I}[\text{correct MMS}]) \quad (۱۳-۵)$$

همبستگی سوم: همبستگی بین مقدار گیت g_{12} و درستی نویسه در نمای Whisper:

$$r_{\text{whisper}} = \text{Corr}(g_{12}, \mathbb{I}[\text{correct Whisper}]) \quad (۱۴-۵)$$

همبستگی چهارم: همبستگی جزئی بین g_{12} و درستی نما، با کنترل اثر موقعیت:

$$r_{xy \cdot z} = \frac{r_{xy} - r_{xz} \cdot r_{yz}}{\sqrt{(1 - r_{xz}^2)(1 - r_{yz}^2)}} \quad (۱۵-۵)$$

که در آن $x = g_{12}$ ، $y = \mathbb{I}[\text{correct view}]$ و $z = \text{position}$.

۵-۴-۲ نتایج همبستگی خام

نتایج همبستگی خام در جدول ۵-۷ آمده است.

جدول ۵-۷: همبستگی خام گیت با موقعیت و درستی نماها

متغیر	همبستگی با g_{12}	p-value	وضعیت
موقعیت کاراکتر	$+0.7744$	< 0.001	معنادار
درستی MMS	-0.2970	< 0.001	معنادار
درستی Whisper	-0.2930	< 0.001	معنادار

در نگاه اول، این نتایج نشان می‌دهد که گیت با هر سه متغیر (موقعیت و درستی هر دو نما) همبستگی دارد. اما این تفسیر نادرست است. دلیل آن، وجود همبستگی هم‌خطی (Collinearity) بین موقعیت و درستی نماها است. جدول ۵-۸ این همبستگی را نشان می‌دهد.

جدول ۵-۸: همبستگی درستی نماها با موقعیت

متغیر	همبستگی با موقعیت
درستی MMS	-0.3448
درستی Whisper	-0.3343

هر دو نما همبستگی منفی معناداری با موقعیت دارند. این یعنی در ادامه جمله، هر دو مدل ASR بدتر می‌شوند. این یافته با شهود زبانی هم‌خوانی دارد: جمله‌های طولانی‌تر، ساختار پیچیده‌تری دارند و مدل‌های ASR در ادامه آن‌ها تمرکزشان را از دست می‌دهند.

از آنجا که هم گیت با موقعیت همبستگی دارد ($r = +0.77$) و هم درستی نما با موقعیت ($r = -0.34$)، یک همبستگی کاذب بین گیت و درستی نماها ایجاد می‌شود.

۵-۴-۳ تحلیل همبستگی جزئی — کشف همبستگی کاذب

برای حذف اثر موقعیت، همبستگی جزئی محاسبه شد. نتایج در جدول ۵-۹ آمده است.

جدول ۵-۹: همبستگی جزئی گیت با درستی نما (کنترل موقعیت)

متغیر	همبستگی خام	همبستگی جزئی	کاهش
درستی MMS (کنترل موقعیت)	-0.2970	-0.504%	$83/03\%$
درستی Whisper (کنترل موقعیت)	-0.2930	-0.573%	$80/44\%$

نتیجه‌گیری قطعی: همبستگی گیت با درستی نما از حدود -0.29 به -0.05 کاهش یافت. این یعنی بیش از 80% از همبستگی ظاهری اولیه، صرفاً محصول جانبی همبستگی گیت با موقعیت بوده است.

۵-۴-۴ تفسیر نهایی و نتیجه‌گیری آزمون

یافته‌های بخش‌های ۵-۴-۲ و ۵-۴-۳ را می‌توان در سه نکته کلیدی خلاصه کرد:

نکته اول — گیت به یک **heuristic** وابسته به موقعیت فروپاشیده است. همبستگی $+0.77$ با موقعیت، نشان می‌دهد که رفتار گیت تقریباً توسط موقعیت کاراکتر در جمله قابل پیش‌بینی است. این heuristic می‌گوید: «در ابتدای جمله وزن بالاتر، در انتها وزن پایین‌تر.»

نکته دوم — گیت هیچ سیگنال محتوایی مستقلی ندارد. همبستگی جزئی -0.05 ، پس از حذف اثر موقعیت، در حد نویز آماری است.

نکته سوم — همبستگی ظاهری گیت با محتوا، یک توهم آماری است. این همبستگی ناشی از همبستگی مشترک هر دو متغیر با موقعیت است، نه یک ارتباط علی مستقیم. فرضیه اصلی این پروژه — که «ترکیب چندنمایی با مکانیزم گیت، می‌تواند به‌طور پویا به نمای مطمئن‌تر اتکا کند» — در زبان فارسی رد می‌شود.

۵-۵ تحلیل نظری: چرا معماری برای فارسی نامناسب است؟

۵-۵-۱ توانایی نظری معماری

از نظر ریاضی، گیت می‌تواند تصمیم محتوایی بگیرد. در هر گام زمانی t ، گیت به اطلاعات زیر دسترسی دارد:

- بردار تعبیه کاراکتر Whisper در موقعیت t : $e_t^{(1)} = \text{Emb}(x_t^{(1)})$

- بردار تعبیه کاراکتر MMS در موقعیت t : $e_t^{(2)} = \text{Emb}(x_t^{(2)})$

- بازنمایی زمینه‌ای BiLSTM از کل جمله: $h_t = \text{BiLSTM}(e_{1:T})$

پس از نظر اطلاعاتی، گیت قادر است ببیند که Whisper «ح» گفته و MMS «ه»، یا Whisper «ذ» گفته و MMS «ز». اگر این تفاوت‌ها سیستماتیک و قابل یادگیری باشند، گیت می‌تواند بر اساس آن‌ها تصمیم بگیرد.

۵-۵-۲ سه مانع عملی

الف) سوگیری استقرایی به سمت راه‌حل‌های ساده: مدل در مسیر کمینه‌سازی loss، ابتدا ساده‌ترین راه‌حل — یعنی تابعی وابسته به موقعیت — را کشف می‌کند. پس از آن، گرادیان‌ها در ناحیه راه‌حل محتوایی بسیار کوچک می‌شوند و مدل هرگز به آن سمت نمی‌رود.

ب) کران اطلاعاتی ظرفیت: مدل کوچک فعلی، از نظر اطلاعاتی قادر نیست نگارش صحیح تمام واژگان فارسی را کدگذاری کند.

ج) عدم ساختار آماری محتوایی در داده: در زبان فارسی، خطاهای هم‌آوا (ذ/ز/ض/ظ، س/ص/ث، ت/ط، ه/ح) الگوی موقعیتی ندارند. هر نویسه ممکن است در هر موقعیتی از هر کلمه‌ای اشتباه شود. بنابراین، حتی اگر مدل

ظرفیت کافی داشت، سیگنال قابل یادگیری وجود نداشت.

۵-۵-۳ تفاوت بنیادی با خط دواناگری

در مقاله مرجع، صریحاً بیان شده که خطاهای Whisper و MMS در زبان راجستانی الگوی موقعیتی و ساختاری دارند. نقل قول مستقیم از مقاله:

«یک فرضیه اغلب ترتیب صحیح صامت‌ها را حفظ می‌کند در حالی که دیگری نشانه‌های محلی نوشتاری مثل nasalization یا طول واکه را درست می‌گیرد، یا مرز کلمه‌ای را نگه می‌دارد که سیستم دیگر ادغام می‌کند.»

این تفاوت‌ها محلی و موقعیتی هستند. اما در فارسی، تفاوت‌ها واژگانی و حافظه‌ای هستند و به کلمه خاص وابسته‌اند، نه به موقعیت در جمله.

جدول ۵-۱۰: مقایسه ماهیت خطاها در دو زبان

ویژگی	راجستانی (دواناگری)	فارسی
محل خطاها	محلی و موقعیتی	واژگانی و حافظه‌ای
الگوی توزیع	سیستماتیک و قابل یادگیری	تصادفی در سطح کاراکتر
قابلیت پیش‌بینی از موقعیت	بالا	پایین
نمونه‌ها	aspiration، matra، مرز کلمه	د/ز/ض/ظ، س/ص/ث، ت/ط، ه/ح
نتیجه برای گیت	می‌تواند تصمیم محتوایی بگیرد	مجبور به heuristic موقعیتی می‌شود

۵-۶ مقایسه با مقاله مرجع

جدول ۵-۱۱ نتایج این پروژه را با مقاله مرجع (زبان راجستانی) مقایسه می‌کند.

جدول ۵-۱۱: مقایسه نتایج با مقاله مرجع

معیار	مقاله مرجع (راجستانی)	این پروژه (فارسی)
CER اولیه Whisper	۱۶/۶۰٪	۹/۸۰٪
CER اولیه MMS	۱۵/۷۳٪	۷/۰۲٪
بهترین CER مدل ترکیبی	۷/۸۶٪	۷/۲۶٪
کاهش CER نسبت به Whisper	۵۲/۶۵٪	۲۵/۹۳٪
کاهش CER نسبت به MMS	۵٪	۳/۴۲٪ -
همبستگی گیت با موقعیت	گزارش نشده	۰/۷۷+
همبستگی جزئی گیت با محتوا	گزارش نشده	۵٪ -

تحلیل: در مقاله مرجع، CER اولیه Whisper حدود ۱۶٪ بود که به ۷/۸۶٪ کاهش یافت — یعنی مدل ترکیبی از هر دو مدل تک‌نمایی بهتر عمل کرد. اما در این پروژه، CER اولیه MMS از قبل ۷/۰۲٪ است و مدل ترکیبی نمی‌تواند این عدد را بهتر کند.

سه دلیل اصلی این تفاوت:

دلیل اول — تفاوت در CER اولیه: در راجستانی، هر دو مدل CER بالایی دارند (۱۵-۱۶٪) که فضای زیادی برای بهبود فراهم می‌کند. در فارسی، MMS از ابتدا CER پایینی دارد (۷٪) و فضای بهبود محدود است.

دلیل دوم — تفاوت در ماهیت خطاها: در راجستانی، خطاها محلی و موقعیتی هستند. در فارسی، خطاها واژگانی و حافظه‌ای هستند. این تفاوت، بنیادی است.

دلیل سوم — تفاوت در ساختار خط: خط دواناگری دارای ساختار قاعده‌مند است و تفاوت‌های نگارشی در آن، الگوهای تکرارشونده ایجاد می‌کند. خط فارسی دارای هم‌آوایی گسترده است و تفاوت‌های نگارشی در آن، الگوهای نزدیک به تصادفی ایجاد می‌کند.

۵-۷ جمع‌بندی

یافته‌های این فصل را می‌توان در پنج نکته کلیدی خلاصه کرد:

نکته اول — معماری ترکیب چندنمایی برای زبان فارسی نامناسب است. مکانیزم گیت — که طراحی آن بر فرض وجود تفاوت‌های موقعیتی بین دو نما بنا شده — در فارسی به یک heuristic ساده وابسته به موقعیت فروپاشی می‌کند. همبستگی $+0.77$ با موقعیت و همبستگی جزئی -0.05 با محتوا، این یافته را تأیید می‌کند.

نکته دوم — علت بنیادی، ماهیت واژگانی خطاهای فارسی است. خطاهای هم‌آوا (ذ/ز/ض/ظ، س/ص/ث، ت/ط، ه/ح) به کلمه خاص وابسته‌اند، نه به موقعیت کاراکتر در جمله. این تفاوت با زبان راجستانی که خطاهایش موقعیتی و ساختاری است، بنیادی است.

نکته سوم — راه‌حل‌های اصلاحی (Gate Variance)، تغییر فرمول، تغییر سطح پردازش) بهبود معناداری ایجاد نکردند. این نشان می‌دهد محدودیت، نه در بهینه‌سازی، نه در ظرفیت، بلکه در عدم تطابق فرض طراحی با ماهیت داده است.

نکته چهارم — کران اطلاعاتی ظرفیت مدل، مانع یادگیری نگارش صحیح واژگان فارسی است. BiLSTM با ۱۲۸ واحد پنهان، ظرفیتی حدود ۱۲۷۰ بیت دارد، در حالی که برای کدگذاری نگارش صحیح ۵۰,۰۰۰ کلمه فارسی، به حدود ۲۵۰,۰۰۰ بیت نیاز است. کمبود حدود ۲۰۰ برابری نشان می‌دهد که مدل کوچک فعلی، از نظر اطلاعاتی قادر به حل این مسئله نیست.

نکته پنجم — رویکردهای محتوایی برای کارهای آینده ضروری است. مدل‌های زبانی بزرگ (LLM)، ویژگی‌های واجی، و مکانیزم‌های توجه در سطح کلمه، گزینه‌های امیدبخش برای تصحیح پس از ASR در زبان فارسی هستند. جمع‌بندی نهایی: این یافته‌ها نشان می‌دهد که موفقیت یک معماری در یک زبان، تضمینی برای موفقیت آن در زبان دیگر نیست. تفاوت‌های خطی و ساختاری زبان‌ها، می‌تواند کارآمدی یک رویکرد را از بهبود در یک

زبان به افت در زبانی دیگر تغییر دهد. این بینش، هم برای پژوهشگران ASR و هم برای طراحان معماری‌های چندزبانه، اهمیت بنیادی دارد.

فصل ششم

جمع‌بندی و پیشنهاد کارهای آینده

۶-۱ مقدمه فصل

در این فصل، یافته‌های حاصل از پژوهش حاضر را جمع‌بندی می‌کنیم و بر اساس آن‌ها، مسیرهای امیدبخش برای پژوهش‌های آینده را پیشنهاد می‌دهیم. همان‌طور که در فصل پنجم نشان داده شد، معماری ترکیب چندنمایی مبتنی بر گیت که برای زبان راجستانی (با خط دواناگری) موفق بوده است، در زبان فارسی به نتیجه مطلوب نمی‌رسد. این یافته، اگرچه در نگاه اول یک «شکست» به نظر می‌رسد، ولی نشان می‌دهد فرض‌های طراحی که در یک خانواده زبانی معتبرند، می‌توانند در خانواده زبانی دیگر کاملاً بی‌اثر باشند.

۶-۲ دستاوردهای فنی

این پروژه در سطح فنی، دستاوردهای زیر را به همراه داشته است:

نخست، پیاده‌سازی کامل یک چارچوب تصحیح پس از ASR. یک معماری رمزگذار-رمزگشا با BiLSTM مشترک، مکانیزم گیت‌دار چندنمایی، و رمزگشای توجهی Bahdanau در محیط PyTorch پیاده و آموزش داده شد. این معماری، تمام اجزای مطرح‌شده در مقاله مرجع را پوشش می‌دهد و به‌عنوان یک baseline قابل استفاده برای پژوهش‌های آینده عمل می‌کند.

دوم، پیاده‌سازی دقیق دو راهبرد هم‌ترازی کاراکتری. راهبرد نخست (هم‌ترازی با کاراکتر تهی) و راهبرد دوم (هم‌ترازی با جایگزینی کاراکتر از دنباله طولانی‌تر) هر دو پیاده‌سازی و ارزیابی شدند. نتایج نشان داد که راهبرد نخست، به دلیل حفظ اطلاعات تمایزی میان دو نما، عملکرد بهتری دارد.

سوم، نرمال‌سازی جامع متن فارسی. یک pipeline کامل شامل هفت مؤلفه (یکسان‌سازی الفبا، نرمال‌سازی یونیکد، یکسان‌سازی ارقام، حذف علائم نگارشی، حذف اعراب، حذف تکرار حروف، یکسان‌سازی فاصله‌ها) به همراه مجموعه‌ای از قواعد Regex برای اصلاح خط‌الرسم اشتباه در مجموعه Common Voice طراحی و پیاده‌سازی شد. این pipeline، خود یک ابزار قابل استفاده در پروژه‌های آینده پردازش زبان فارسی است.

چهارم، پیاده‌سازی Gate Variance Regularization و تکنیک‌های تقویت پویایی گیت. مجموعه‌ای از اصلاحات (شامل Scaled Sigmoid، حذف Layer Normalization، Sensory Dropout، کامل و جزئی، و افزودن جمله واریانس به تابع هزینه) پیاده‌سازی و ارزیابی شد. اگرچه این اصلاحات منجر به بهبود مدل نشد اما تحلیل آنها منجر به درک نحوه عملکرد گیتها شد.

پنجم، طراحی و اجرای یک آزمون آماری جامع. یک آزمون فرضیه دو مرحله‌ای (شامل همبستگی خام و همبستگی جزئی) روی ۵۷,۹۵۸ نقطه داده طراحی و اجرا شد که توانست فرضیه «تصمیم‌گیری محتوایی گیت» را در زبان فارسی رد کند.

۳-۶ محدودیت‌های پژوهش

این پژوهش، مانند هر پژوهش دیگری، محدودیت‌هایی دارد که ذکر آن‌ها برای شفافیت علمی ضروری است: محدودیت اول — اندازه مدل. مدل به کاررفته در این پروژه، یک مدل کوچک با ۲/۷ میلیون پارامتر است. این اندازه، برای نشان دادن توانایی‌های معماری‌های بزرگ‌تر (مانند LLMها) کافی نیست. یافته‌های ما نشان می‌دهد که مدل کوچک، از نظر اطلاعاتی قادر به حل مسئله نیست، اما این نتیجه ممکن است برای مدل‌های بزرگ‌تر صادق نباشد.

محدودیت دوم — حجم داده. مجموعه Common Voice فارسی با حدود ۴۹,۰۰۰ نمونه، در مقایسه با زبان‌های پرمبوع مانند انگلیسی که میلیون‌ها ساعت داده دارند، کوچک است. با داده بیشتر، ممکن است مدل بهتری آموزش پیدا کند.

محدودیت سوم — استفاده از دو نما. این پروژه از دو نمای Whisper و MMS استفاده کرد. در آینده، می‌توان از خروجی چند مدل ASR مختلف (شامل مدل‌های فارسی اختصاصی) بهره برد که ممکن است الگوهای خطای متفاوتی داشته باشند.

محدودیت چهارم — عدم استفاده از ویژگی‌های واجی. مدل فعلی صرفاً بر کاراکترها عمل می‌کند و از اطلاعات واجی یا آوایی که می‌تواند برای تمایز نویسه‌های هم‌آوا مفید باشد، بهره نمی‌برد.

محدودیت پنجم — عدم بررسی لهجه‌ها. مجموعه Common Voice شامل لهجه‌های مختلف فارسی است، اما در این پروژه تحلیل جداگانه‌ای بر حسب لهجه انجام نشد.

۴-۶ پیشنهاد کارهای آینده

بر اساس یافته‌های این پژوهش، مسیرهای زیر برای کارهای آینده پیشنهاد می‌شوند. این پیشنهادها به ترتیب اولویت ارائه می‌شوند.

۱-۴-۶ استفاده از مدل‌های زبانی بزرگ

مهم‌ترین پیشنهاد برای کارهای آینده، استفاده از LLM‌های تنظیم‌شده (Fine-tuned) برای تصحیح پس از ASR در زبان فارسی است. دلایل این پیشنهاد:

دلیل اول — توانایی درک زمینه. LLM‌ها به دلیل آموزش روی حجم عظیمی از متن، توانایی درک زمینه و تشخیص کلمات هم‌آوا را دارند. برای مثال، یک LLM می‌تواند با دیدن جمله «... حضرت ...» تصمیم بگیرد که «ح» درست است، نه «ه»، چون این ترکیب در متن‌های آموزشی خود دیده است.

دلیل دوم — ظرفیت اطلاعاتی بالا. LLM‌ها دارای میلیاردها پارامتر هستند که ظرفیت اطلاعاتی آن‌ها را به مراتب بیشتر از مدل کوچک فعلی می‌کند. این ظرفیت، برای کدگذاری نگارش صحیح واژگان فارسی کافی است.

دلیل سوم — انعطاف‌پذیری در تنظیم دقیق. LLM‌ها را می‌توان با داده‌های خاص فارسی (شامل جفت‌های «خروجی ASR - متن صحیح») تنظیم دقیق کرد و به نتایج بهتری دست یافت.

۲-۴-۶ ویژگی‌های واجی

پیشنهاد دوم، افزودن ویژگی‌های واجی به مدل است. در زبان فارسی، چندین نویسه مختلف دارای تلفظ یکسان هستند (د/ذ/ض/ظ، س/ص/ث، ت/ط، ه/ح). اگر به جای کاراکتر، ویژگی واجی (مثل Phoneme) به مدل داده شود، مدل می‌تواند تشخیص دهد که این نویسه‌ها یکسان تلفظ می‌شوند و تفاوت آن‌ها صرفاً در نگارش است.

پیاده‌سازی پیشنهادی: یک embedding واجی به مدل اضافه شود که هر کاراکتر را به واج متناظرش نگاشت کند. این embedding می‌تواند با embedding کاراکتری ترکیب شود و به عنوان ورودی به BiLSTM داده شود.

۶-۴-۳ معماری ترنسفورمر

پیشنهاد چهارم، استفاده از معماری Transformer به جای LSTM است. اگرچه در این پروژه از BiLSTM استفاده شد، اما معماری‌های Transformer به دلیل Self-Attention، توانایی بهتری در درک زمینه دارند. به‌ویژه، Multi-Head Attention می‌تواند به‌طور همزمان به اطلاعات محلی (کاراکترهای همسایه) و اطلاعات جهانی (زمینه کل جمله) توجه کند.

۶-۵ پیوست ریاضی: تحلیل کران اطلاعاتی

در این بخش، به‌طور دقیق اثبات می‌کنیم که مدل کوچک به‌کاررفته در این پروژه، از نظر اطلاعاتی قادر به یادگیری نگارش صحیح واژگان فارسی نیست. این استدلال، بر پایه نظریه اطلاعات (Information Theory) بنا شده است.

۶-۵-۱ ظرفیت اطلاعاتی BiLSTM

یک BiLSTM با H واحد پنهان در هر جهت، در هر گام زمانی یک بردار $h_t \in \mathbb{R}^{2H}$ تولید می‌کند. هر مؤلفه این بردار، یک عدد اعشاری با دقت محدود است. اگر فرض کنیم که هر مؤلفه می‌تواند به‌طور مؤثر $\log_2(N)$ بیت اطلاعات را کدگذاری کند (که N تعداد سطوح دقت است، معمولاً $N \approx 1000$)، ظرفیت کل مدل به‌صورت زیر محاسبه می‌شود:

$$C_{\text{BiLSTM}} = 2H \cdot \log_2(N) \approx 256 \cdot \log_2(1000) \approx 256 \cdot 10 \approx 2560 \text{bits} \quad (۶-۱)$$

با در نظر گرفتن این که بخشی از این ظرفیت صرف کدگذاری اطلاعات زمینه‌ای (نه واژگانی) می‌شود، ظرفیت مؤثر برای کدگذاری واژگان حدود $C_{\text{eff}} \approx 1270$ بیت است.

۶-۵-۲ اطلاعات لازم برای نگارش صحیح واژگان فارسی

برای یادگیری نگارش صحیح واژگان فارسی، مدل باید بتواند برای هر کلمه، نگارش صحیح آن را از میان چندین گزینه ممکن انتخاب کند. فرض کنید:

• $|V|$: تعداد کلمات رایج در زبان فارسی (حدود ۵۰,۰۰۰)

• A : اندازه الفبای فارسی (حدود ۳۲ حرف)

• L : میانگین طول یک کلمه (حدود ۵ حرف)

اطلاعات لازم برای کدگذاری نگارش صحیح یک کلمه w به صورت زیر است:

$$I(w) = \log_2(A^L) = L \cdot \log_2(A) = 5 \cdot \log_2(32) = 5 \cdot 5 = 25\text{bits} \quad (۲-۶)$$

اطلاعات لازم برای کدگذاری نگارش صحیح تمام واژگان به صورت زیر محاسبه می شود:

$$I_{\text{total}} = |V| \cdot I(w) = 50000 \cdot 25 = 1,250,000\text{bits} \quad (۳-۶)$$

۳-۵-۶ مقایسه و نتیجه گیری

با مقایسه ظرفیت مؤثر مدل و اطلاعات لازم، به نتیجه زیر می رسیم:

$$\frac{I_{\text{total}}}{C_{\text{eff}}} = \frac{1,250,000}{1270} \approx 985 \quad (۴-۶)$$

این نسبت نشان می دهد که اطلاعات لازم برای یادگیری نگارش صحیح واژگان فارسی، حدود ۱۰۰۰ برابر ظرفیت مؤثر مدل کوچک فعلی است. اگر محاسبات را با فرض های محافظه کارانه تر (مثلاً $|V| = 10000$ و $L = 3$) انجام دهیم، نسبت به حدود ۱۰۰ کاهش می یابد، اما همچنان دو مرتبه بزرگی (۱۰۰ برابر) بیشتر از ظرفیت مدل است.

نتیجه: حتی با فرض های محافظه کارانه، مدل کوچک فعلی (با ۱۲۸ واحد پنهان) از نظر اطلاعاتی قادر نیست نگارش صحیح واژگان فارسی را یاد بگیرد. این یافته، یکی از دلایل اصلی شکست رویکرد ترکیب چندنمایی در زبان فارسی است.

پیامد عملی: برای حل این مسئله، نیازمند یک مدل با ظرفیت اطلاعاتی حداقل ۱۰۰ برابر بیشتر هستیم. این مقیاس ظرفیت، در خانواده مدل های LLM (با میلیاردها پارامتر) قابل دسترسی است. بنابراین، استفاده از LLM برای تصحیح پس از ASR در زبان فارسی، نه یک انتخاب سلیقه ای، بلکه یک ضرورت اطلاعاتی است.

مراجع

- [1] Bhandari, A., and Harit, G., “Post-ASR Correction for Low-Resource Rajasthani Language”, *ACM Transactions on Asian and Low-Resource Language Information Processing*, Vol. 25, No. 3, pp. 1–13, 2026.
- [2] Aliyazdi, A., “Common Voice 22.0 – Persian (fa) Dataset”, *Hugging Face*.

